

University of the Philippines - Diliman Campus
05/29/2026

Low-Resource Sentiment Analysis on the 2025 Steam Dataset Using HNSW and c-TF-IDF

Reducing the number of dimensions from 1024 to 4 + 5 using feature extraction, and achieved 93.49% macro-averaged f1-score on a 4:1 positive-negative review ratio with 550k samples

PRESENTED BY:
Jethro Elijah F. Bolima

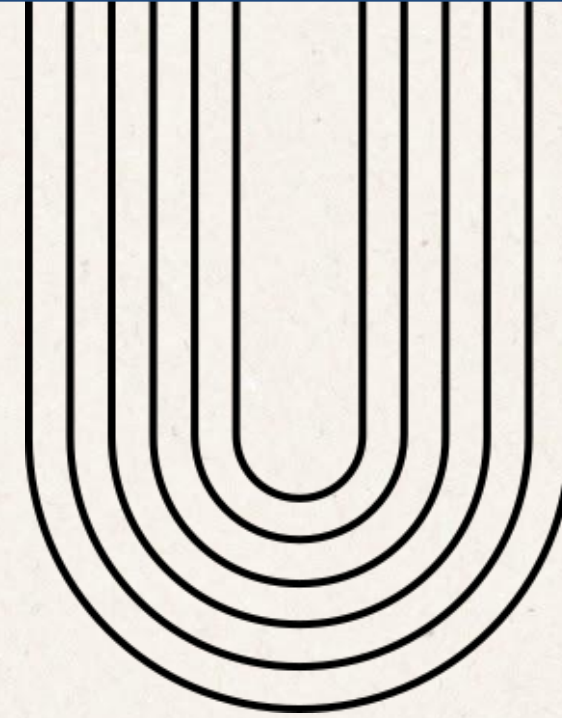


Agenda

01	Problem of the Study
02	Context and Background
03	Connecting BGE-M3 text embeddings with HNSW and C-TF-IDF
04	Reducing dimensions with Incremental PCA
05	Fine-tuning embedding models and creating my own text embeddings
06	Topic Modelling on Positive and Negative Reviews
07	Synthesis
08	Conclusion

Problem of the Study

To put it simply...



Problem # 1

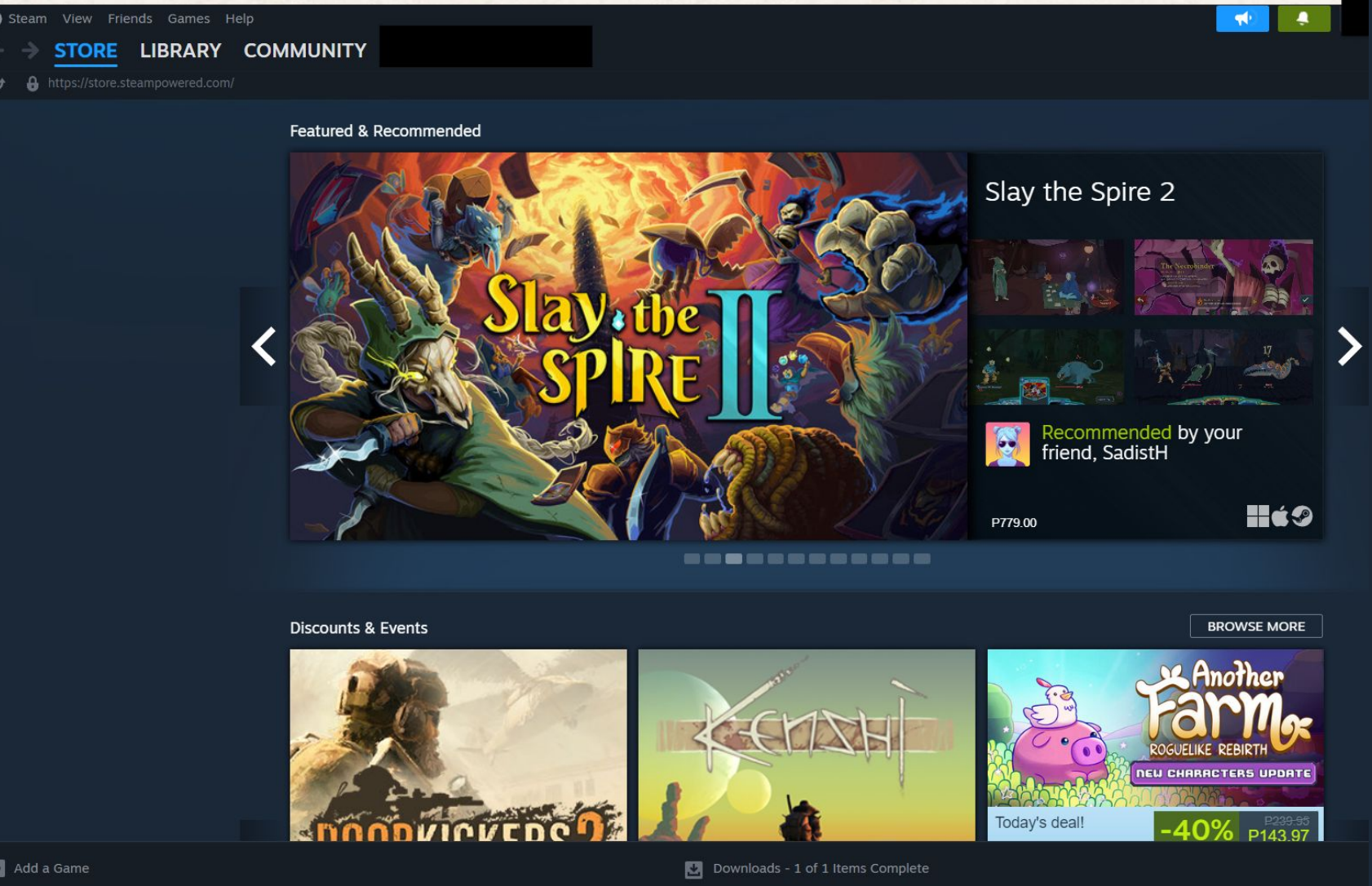
How can a system equipped with 16 GB RAM and 4 GB VRAM efficiently compute a processed dataset of 544,744 samples with 1,027 total dimensions, including 3 behavioral metrics?



Problem # 2

Which specific aspects of a game are driving player sentiment in these reviews?

Context and Background



Steam is a distribution platform of games. You can leave reviews on games that you have played.



Context and Background

Enter **Donald Fountain**, the creator of The Steam Dataset 2025:
A Large-Scale, Multi-Modal Dataset of the Steam Gaming Platform

Metric	Value	Coverage
Total Applications	239,664	Complete accessible Steam catalog
User Reviews	1,048,148	Full review corpus with metadata
Developers	54,321	Complete developer ecosystem
Publishers	39,876	Full publisher network
Temporal Range	1997-2025	28 years of platform evolution
Success Rate	56%	Transparent quality metrics

Scope of the Study:

**English Reviews and
emoticon-only reviews are
allowed.**

- 544,744 samples (after cleaning)
24.50% negative-to-total ratio
- 133,443 Negative Reviews
 - 411,301 Positive Reviews

Context and Background

We will be using the following:

Target

- **voted_up** (positive or negative review)

Features

- **Embedding Table**
 - **review_embedding** (OOTB BGE-M3 text embeddings)
- **Text Table**
 - **review_text** (actual reviews)
 - **votes_up** (no. of votes the review has for being helpful)
 - **votes_funny** (no. of votes the review has for being funny)
 - **author_playtime_at_review**
- **recommendationid** (for joining the tables, dropped later)

DOI 10.5281/zenodo.17266922

reviews	
Columns (25)	
recommendationid	text
appid	bigint
author_steamid	text
author_num_games_owned	bigint
author_num_reviews	bigint
author_playtime_forever	bigint
author_playtime_last_two_weeks	bigint
author_playtime_at_review	bigint
author_last_played	bigint
language	text
review_text	text
timestamp_created	bigint
timestamp_updated	bigint
voted_up	boolean
votes_up	bigint
votes_funny	bigint
weighted_vote_score	numeric(10,8)
comment_count	bigint
steam_purchase	boolean
received_for_free	boolean
written_during_early_access	boolean
created_at	timestamp with time zone
updated_at	timestamp with time zone
embedding_run_id	bigint
review_embedding	vector(1024)

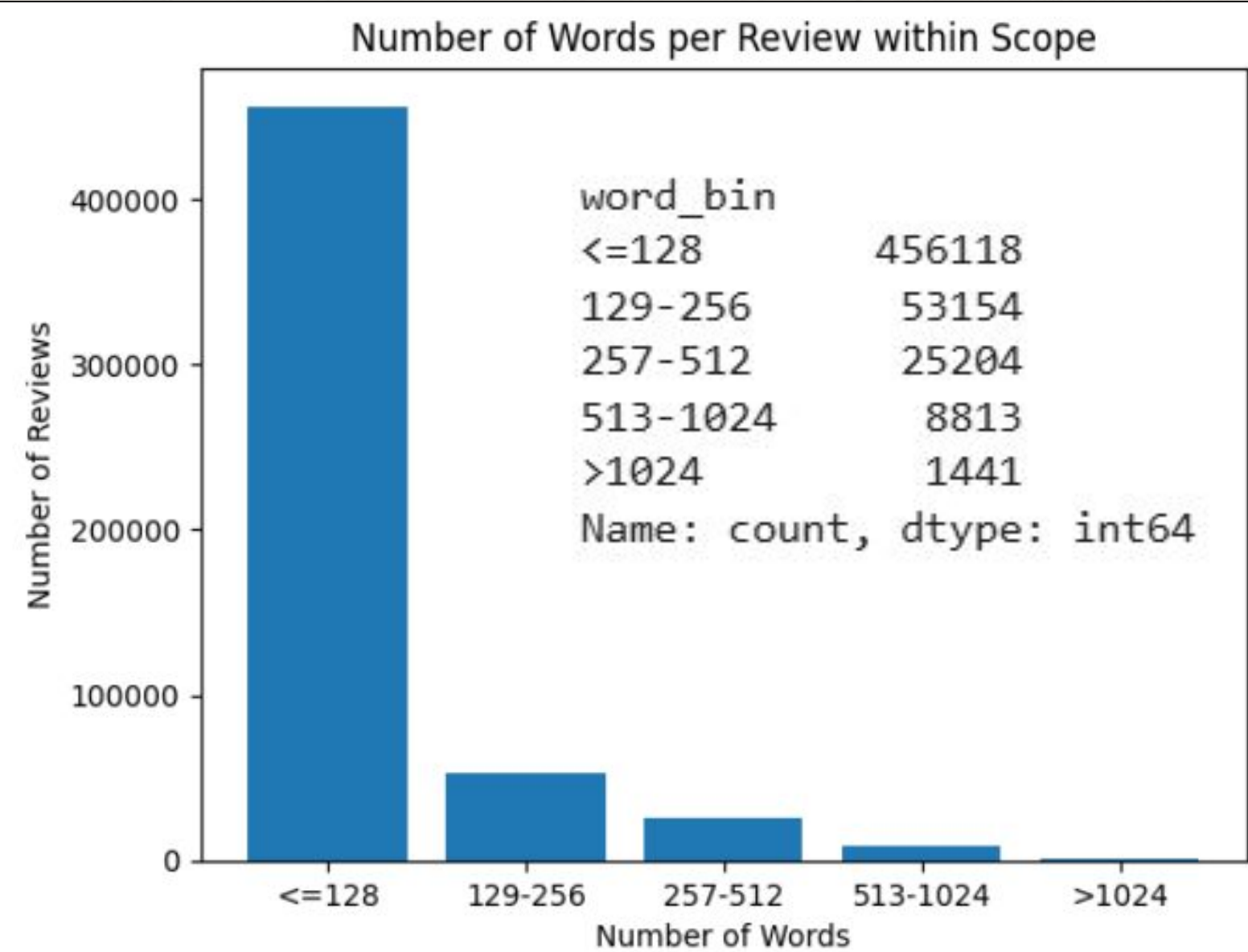
Context and Background

What is BGE-M3 and why did the author use it?

Beijing Academy of Artificial Intelligence **G**eneral **E**mbeddings - **M**ulti-Linguality, **M**ulti-Functionality, and **M**ulti-Granularity

Model Specifications

Specification	Value	Impact
Base Model	XLM-RoBERTa	Proven multilingual foundation
Parameters	569 million	Optimal for 24GB GPU
Model Size	~2.3GB	Fits in VRAM with batch processing
Max Tokens	8192	Handles long descriptions
Output Dims	1024	Standard semantic search dimension
Normalization	L2 normalized	Ready for cosine similarity



<https://github.com/vintagedon/steam-dataset-2025/blob/main/docs/methodologies/vector-embeddings.md>

With our scope, improvements can be made.

Context and Background

So we have to process **544,744** reviews with...

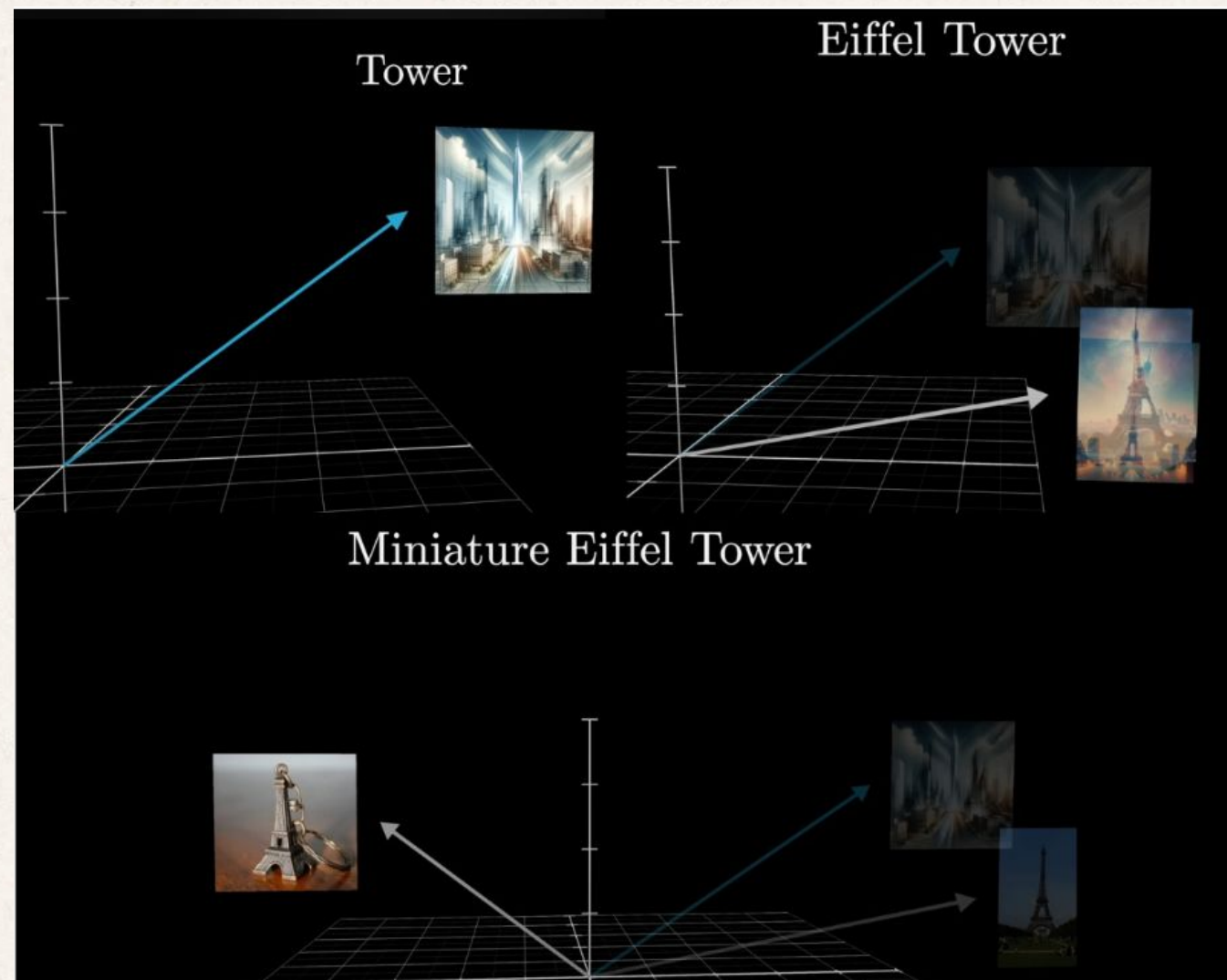
- **1,024** dimensions of out-of-the-box (OOTB) text embeddings
- **3** behavioral metrics (votes_up, votes_funny, author_playtime_at_review)
- somehow extract features from the text themselves

On a laptop with 4 GB VRAM and 16 GB RAM.

HOW?

Connecting BGE-M3 text embeddings with HNSW and C-TF-IDF

A text embedding is essentially a list of coordinates of **a single point** in a high-dimensional space



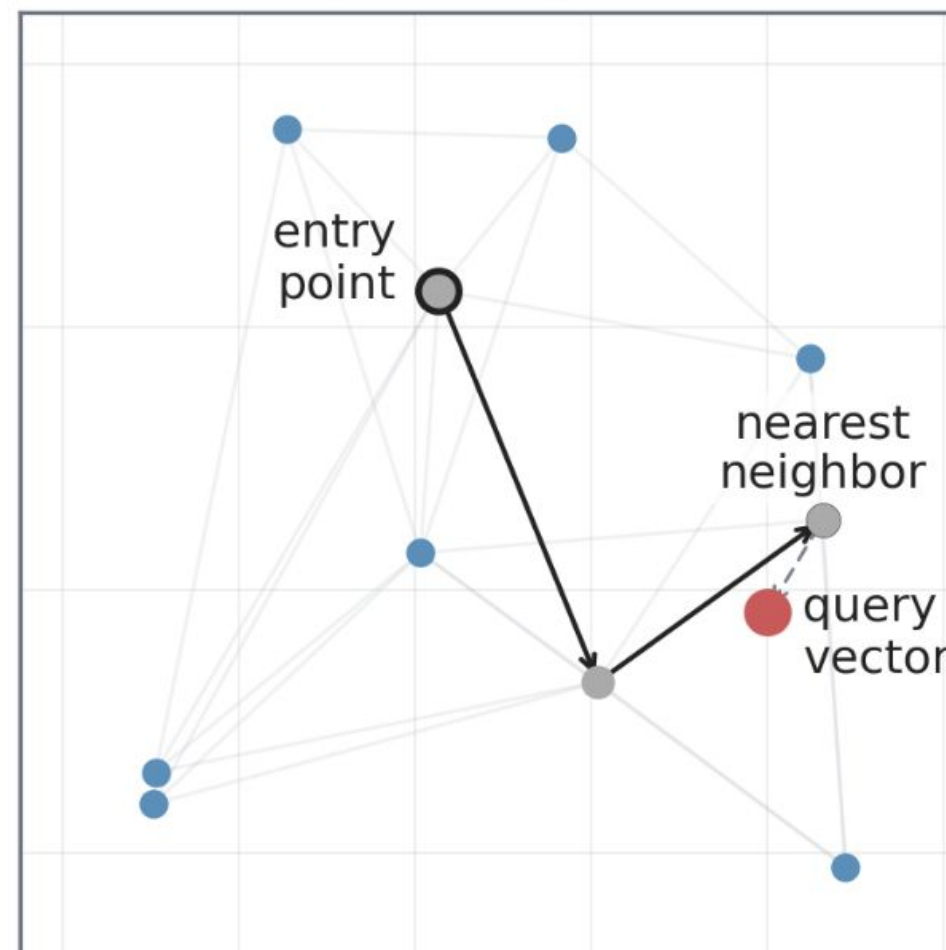
Clustering is a possibility, but KNN on a 550k by 1024 dataset is very slow, uses too much memory, and does not perform well on high dimensions (computes distance vs every point in the dataset every query)....

so we use approximate nearest neighbors.

Connecting BGE-M3 text embeddings with HNSW and C-TF-IDF

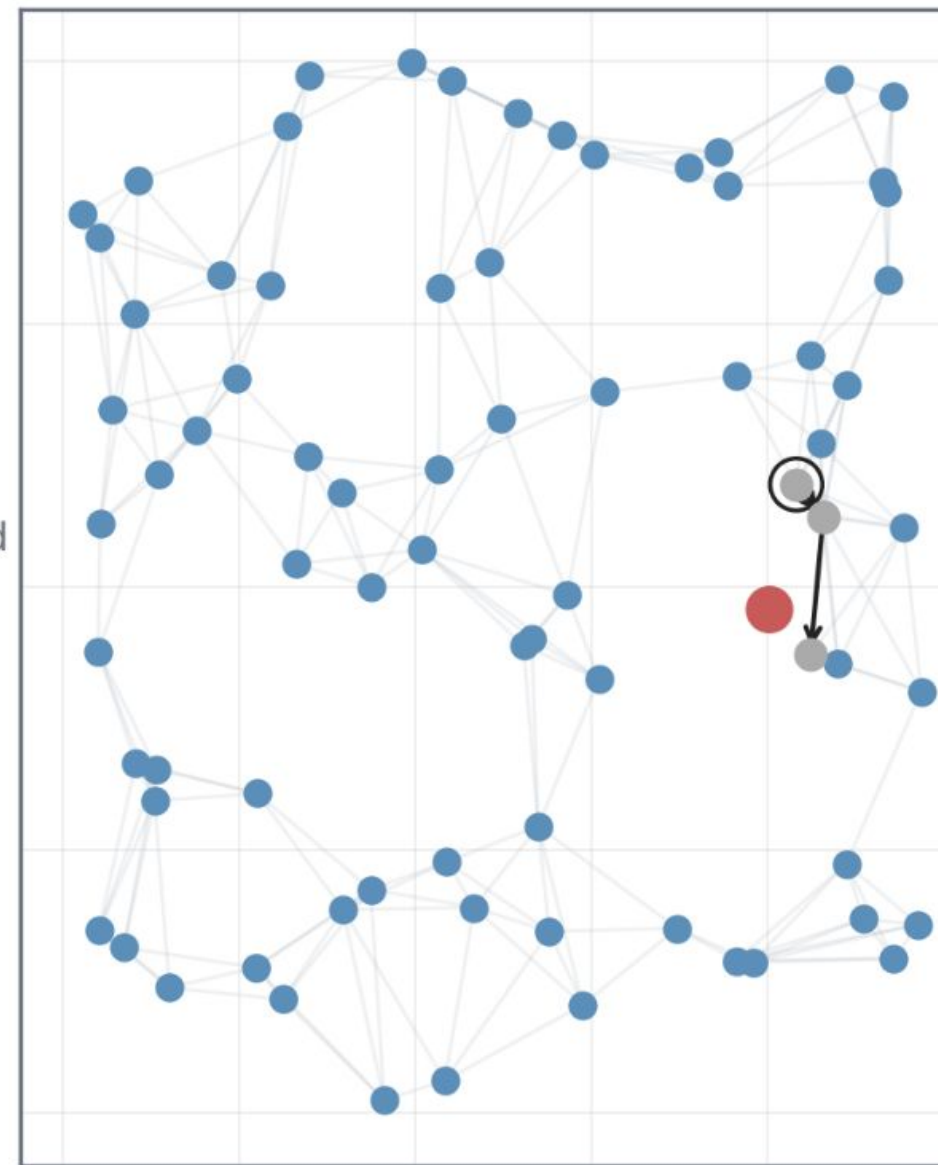
Hierarchical Navigable Small World (HNSW)

Layer 2 (Sparse)



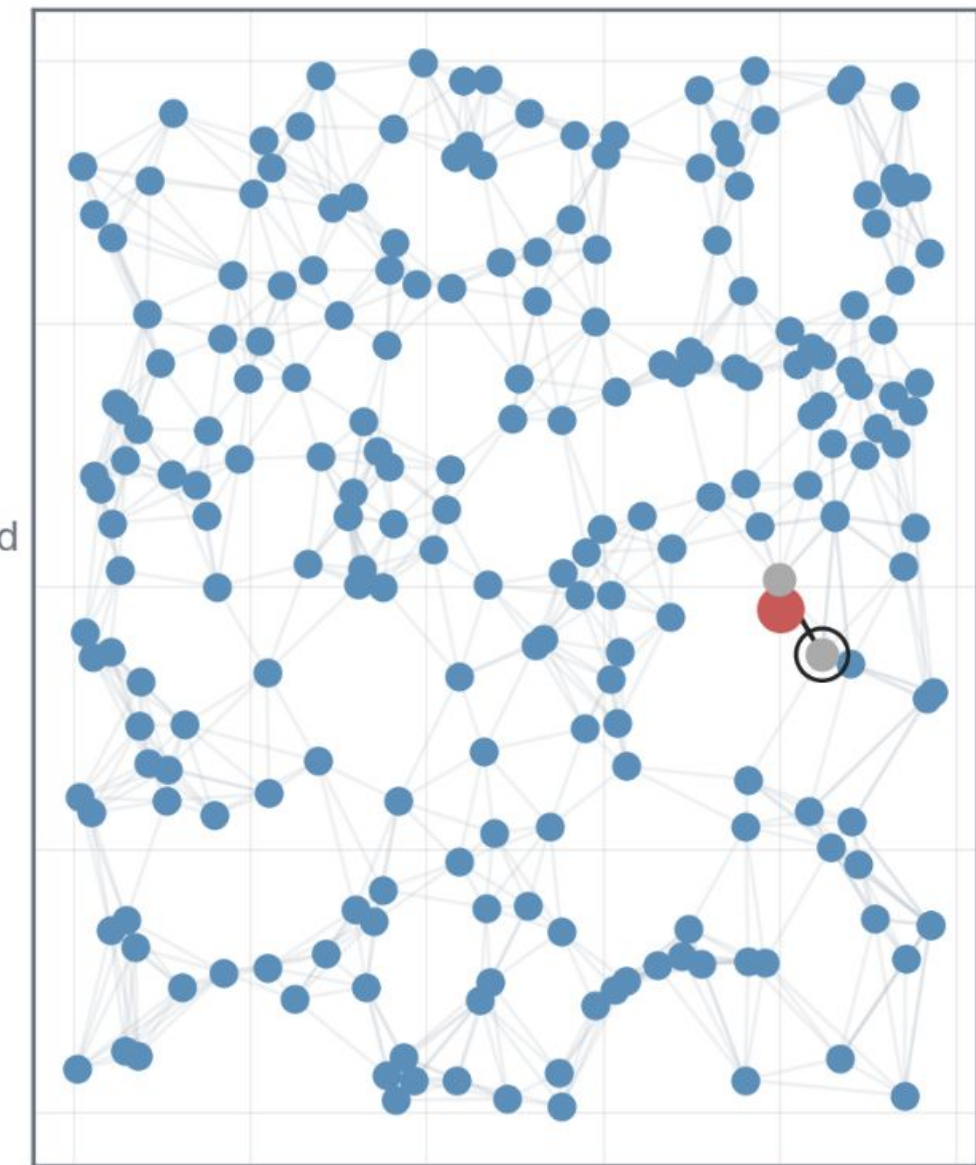
descend
→

Layer 1 (Intermediate)



descend
→

Layer 0 (Dense — All Points)



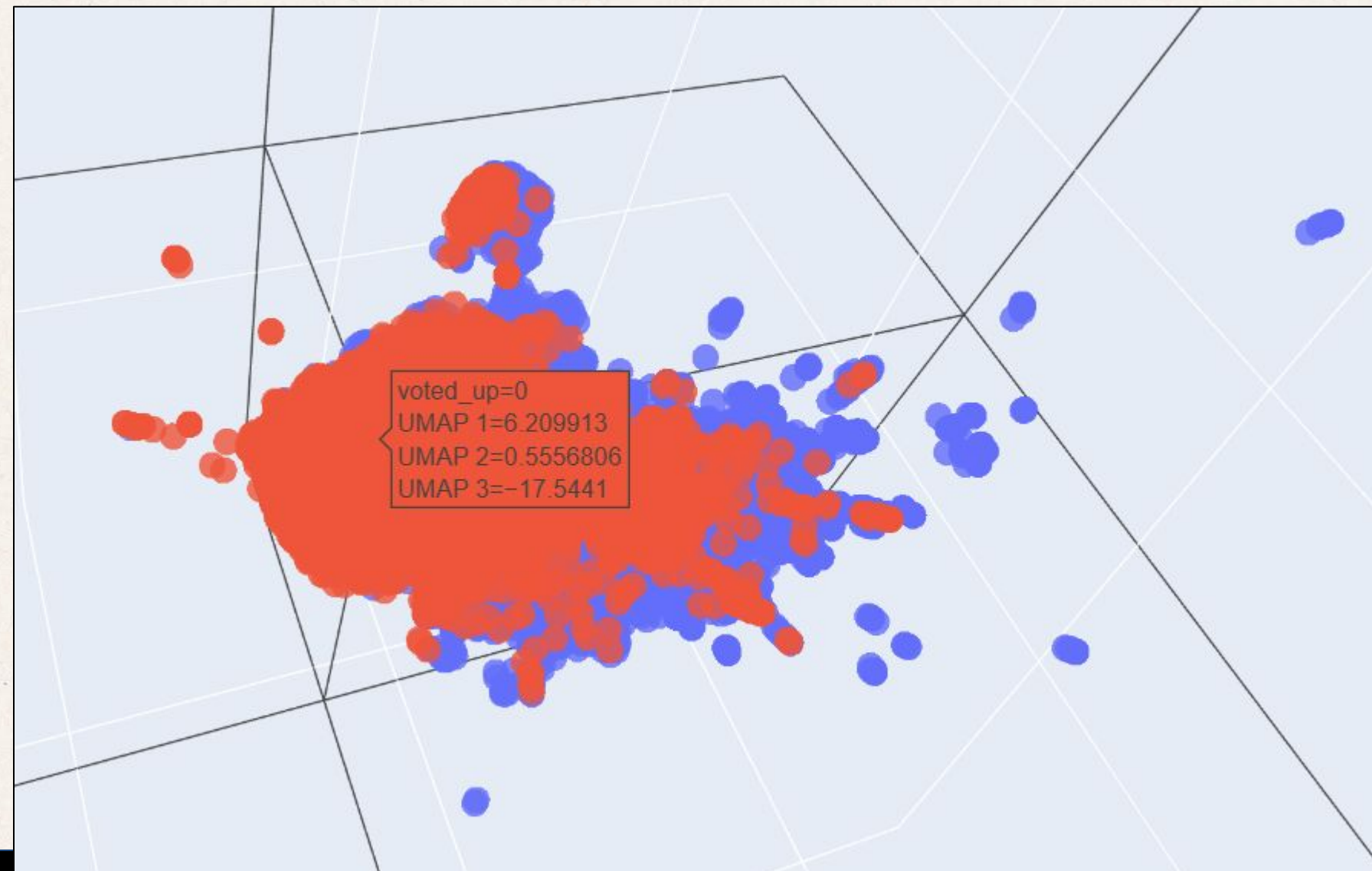
Time Complexity: $O(\log N)$

<https://www.mongodb.com/resources/basics/hierarchical-navigable-small-world>

Connecting BGE-M3 text embeddings with HNSW and C-TF-IDF

With HNSW, we can now get:

- The **distance** between the query and its closest neighbors
- The **indices** of the closest neighbors



At 50k samples, there is a distinguishable separation between positive and negative reviews.


```
[56]: knn_distances.shape
```

```
[51]: knn_distances[0]
```

```
[52]: knn_indices[0]
```

```
[67]: Y_bge_m3[knn_indices][0]
```

[illegible]

Connecting BGE-M3 text embeddings with HNSW and C-TF-IDF

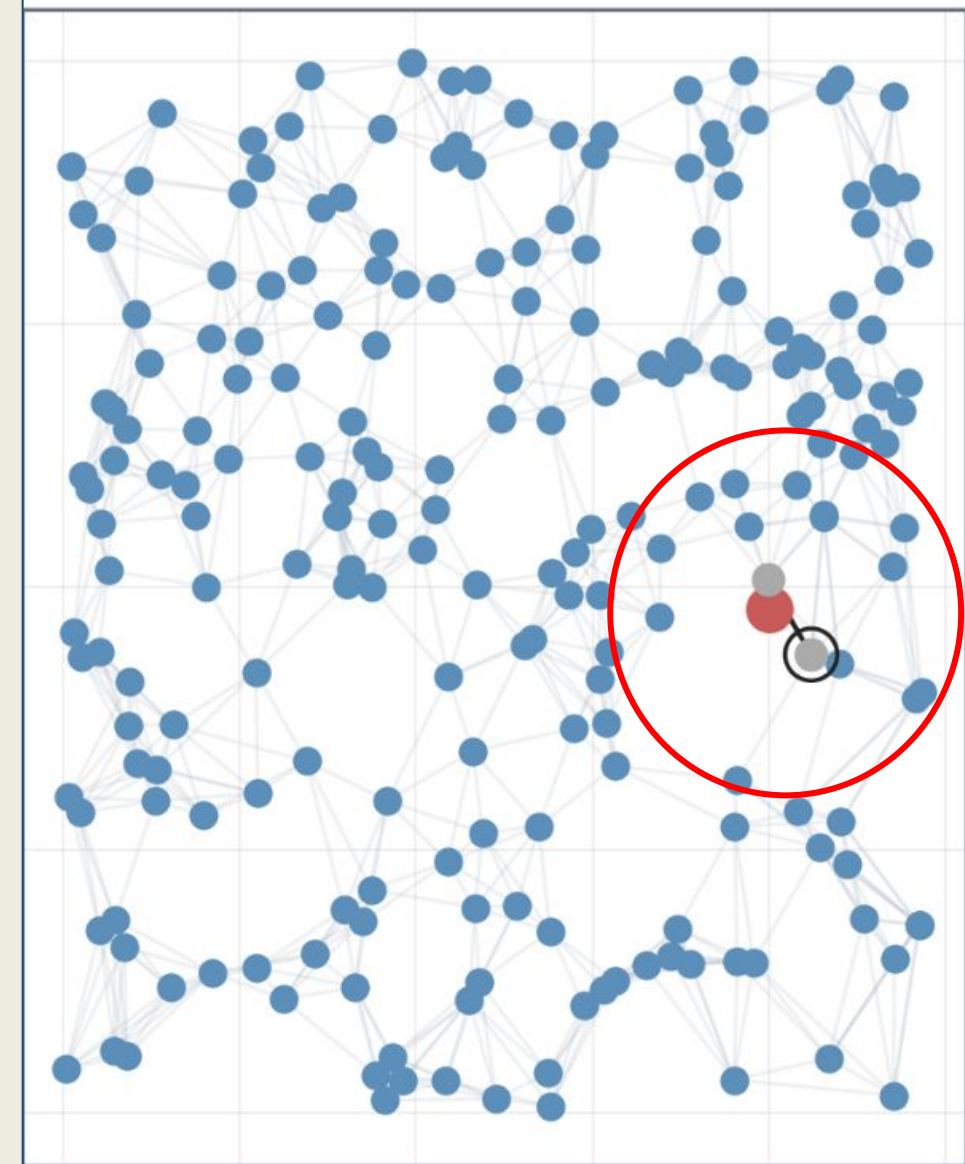
Hierarchical Navigable Small World (HNSW)

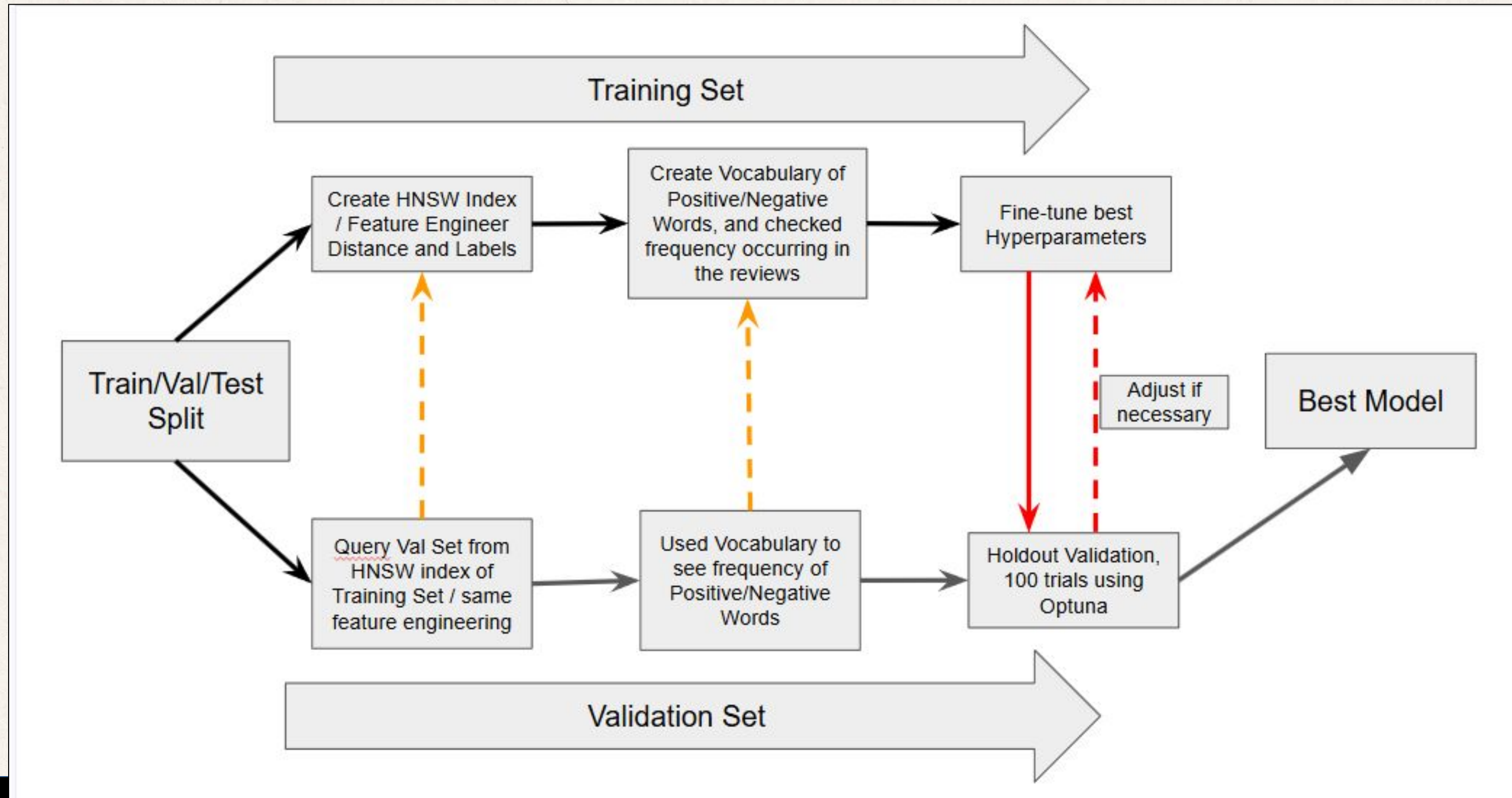
We can represent a point by only looking at its closest neighbors!

- **knn_mean_distances** how tight the neighborhood is (bigger implies being an outlier review)
- **knn_std_distances** how consistent the review is with its neighbors (bigger implies inconsistency)
- **knn_target_encoding_mean** **y_train data only**, check the average sentiment of the y_train neighborhood (higher implies positive)
- **knn_target_encoding_std** = **y_train data only**, check the sentiment consistency of the y_train neighborhood

From 1024 dimensions into just 4 features

Layer 0 (Dense — All Points)





Connecting BGE-M3 text embeddings with HNSW and C-TF-IDF

Class-based Term Frequency - Inverse Document Frequency

- OOTB Embedding Models have no domain knowledge
- C-TF-IDF helps in classifying which n-grams are more important in positive and negative reviews
- We will put the “positive” n-grams and “negative” n-grams in separate vocabularies to count the number of occurrences these terms are used in a review. (More positive n-grams imply a positive review)

Connecting BGE-M3 text embeddings with HNSW and C-TF-IDF

C-TF-IDF: Top 5% Positive n-grams, 4015 total

```
[5]: top_positive_words_filtered[:1000]
```

```
[5]: array(['strategies the', 'enthusiasts and', 'long after',  
          'started playing', 'buy for', 'will need', 'challenging with',  
          'happen next', 'and tense', 'get quite', 'unique style', 'at good',  
          'ramps up', 'check if', 'visuals are', 'helps to', 'the vibe',  
          'good bit', 'of genres', 'good pace', 'tear', 'varying', 'tale of',  
          'addiction', 'quite enjoyed', 'are charming', 'best experience',  
          'be difficult', 'backstories', 'recommend everyone', 'done great',  
          'really gives', 'capturing the', 'builds on', 'are satisfying',  
          'visually the', 'pressure', 'and bosses', 'everything works',  
          'and narrative', 'definitely enjoyed', 'or sequel',  
          'shines through', 'adrenaline pumping', 'amazing atmosphere',  
          'relaxing gameplay', 'especially love', 'complexity to',  
          'classic rpg', 'chill out', 'customization', 'worry about',  
          'first time', 'pokemon', 'the pain', 'the simplicity',  
          'and finding', 'better love', 'think outside', 'fascinated',  
          'weaknesses', 'very detailed', 'players engaged', 'man', 'cup of',  
          'creating this', 'appreciation', 'janky but', 'intimidating',  
          'blew mv', 'perfectly complements', 'great expansion', 'an era',
```


Connecting BGE-M3 text embeddings with HNSW and C-TF-IDF

C-TF-IDF: Top 5% Negative n-grams, 4015 total

```
[6]: top_negative_words_filtered[:1000]
```

```
[6]: array(['not worth', 'refund', 'terrible', 'boring', 'waste', 'unplayable',
          'asset', 'not buy', '1001 jigsaw', 'gamers', '1001', 'worst',
          'poor', 'waste of', 'awful', 'broken', 'defects', 'not recommend',
          'garbage', 'the worst', 'poorly', 'gamers to', 'abandoned',
          'pc gamers', 'mobile app', 'jigsaw', 'for gamers', 'solitaire',
          'scam', 'money', 'horrible', 'waste your', 'url developed',
          'don buy', 'bad', 'not fun', 'no way', 'worse', 'trash',
          'asset flip', 'app', 'cannot recommend', 'low quality',
          'the defects', 'impossible to', 'unity com', 'assetstore unity',
          'https assetstore', 'com packages', 'doesn', 'just not',
          'poor quality', 'don waste', 'technical defects', 'mobile',
          'unfortunately', 'assetstore', 'your money', 'just', '000 games',
          'defects push', 'basic minimum', 'uninstalling', 'refunded',
          'to like', 'cash grab', 'peak player', 'can recommend',
          'meet basic', 'steamdb shows', 'occurs once', 'presumably someone',
          'someone loading', 'quickly uninstalling', 'review steamdb',
          'activity occurs', 'million gamers', 'overwhelming lack',
          'month presumably', 'defects and', 'impossible', 'buggy',
```

4 of 4 people (100%) found this review helpful
7 people found this review funny



nuclear shot enthusiast
332 products in account
19 reviews



Not Recommended
0.1 hrs on record

POSTED: 6 FEBRUARY

Product received for free

i want a refund

Was this review helpful?

Yes

No

Funny

Connecting BGE-M3 text embeddings with HNSW and C-TF-IDF

So we have to process **544,744** reviews with...

- **4** features utilizing nearest neighbor distance aggregates and target encoding
- **2** features for relating to domain (positive and negative n-grams)
- **3** behavioral metrics (votes_up, votes_funny, author_playtime_at_review)

We now only have 9 features!

Connecting BGE-M3 text embeddings with HNSW and C-TF-IDF

Baseline (using only 1024 dimensions)

Establishing baseline - OOTB LightGBM on the BGE-M3 embeddings

```
LGBMClassifier_baseline = LGBMClassifier()
LGBMClassifier_baseline.fit(X_train_baseline, y_train)
y_pred_baseline = LGBMClassifier_baseline.predict(X_test_baseline)
print(f1_score(y_test, y_pred_baseline, average = "macro"))
```

```
[LightGBM] [Info] Number of positive: 329041, number of negative: 106754
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 5.133117 seconds.
You can set `force_col_wise=true` to remove the overhead.
```

```
[LightGBM] [Info] Total Bins 261120
[LightGBM] [Info] Number of data points in the training data: 1396591
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.232457, navg=0.767543
[LightGBM] [Info] Start training from score 1.12561
0.8931558381487521
```

Establishing baseline - Balanced Logistic Regression w/ SAGA on the BGE-M3 embeddings

```
LogisticRegression_baseline = LogisticRegression(solver = "saga", class_weight = "balanced")
LogisticRegression_baseline.fit(X_train_baseline, y_train)
y_pred_baseline = LogisticRegression_baseline.predict(X_test_baseline)
print(f1_score(y_test, y_pred_baseline, average = "macro"))
```

```
0.8869953629125011
```

**We need to be
competitive vs
89.32%**

Connecting BGE-M3 text embeddings with HNSW and C-TF-IDF

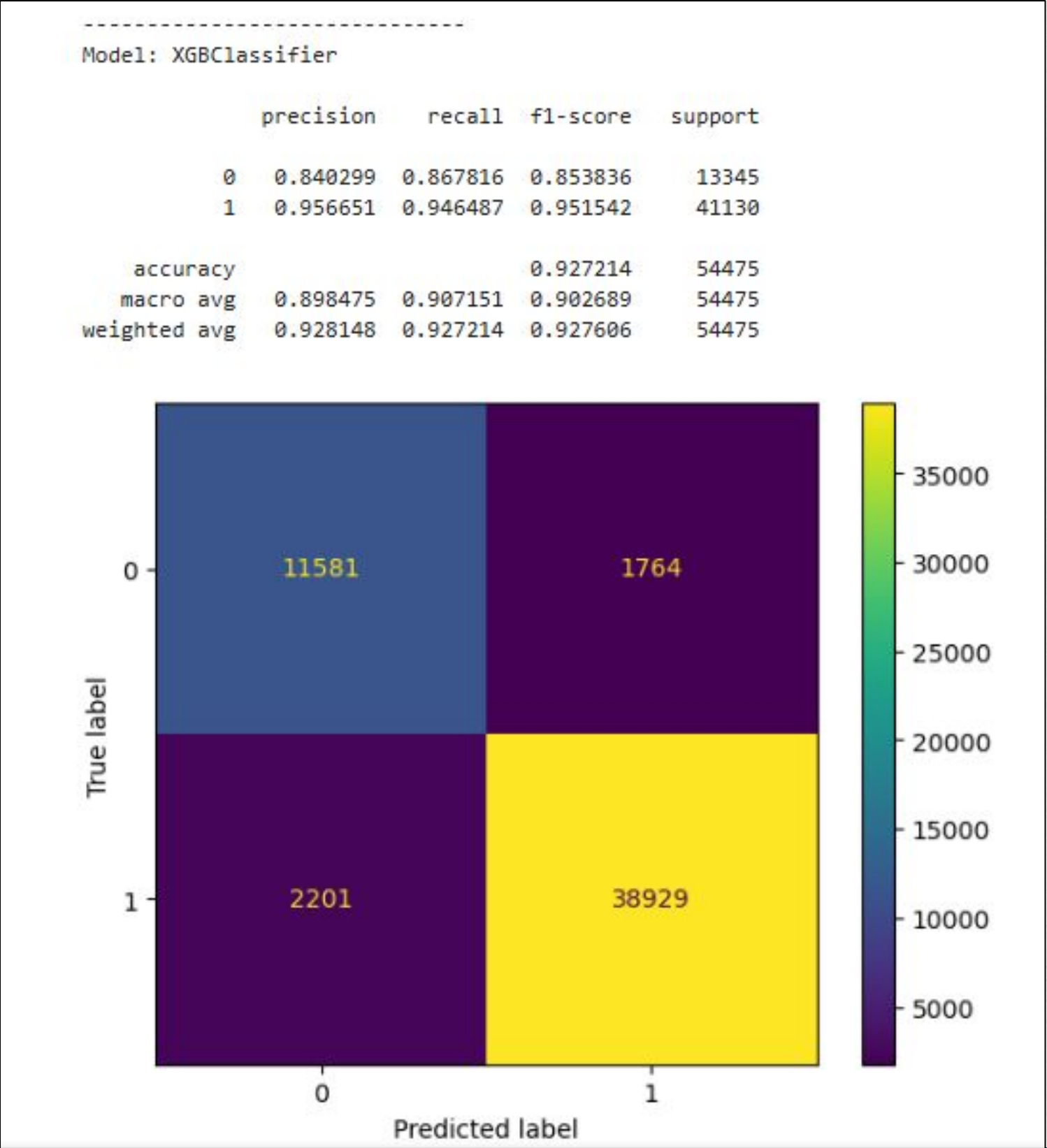
Results

Highest: 90.27%
Lowest: 89.79%

```
[103]: score_test_df.to_parquet("score_test_df_BGE_M3.parquet")
print(score_test_df)
```

	model	macro f1 score - test \
0	XGBClassifier	0.902689
1	LGBMClassifier	0.902365
2	MLPClassifier	0.902246
3	HistGradientBoostingClassifier	0.901705
4	LogisticRegression	0.899042
5	GaussianNB	0.897925

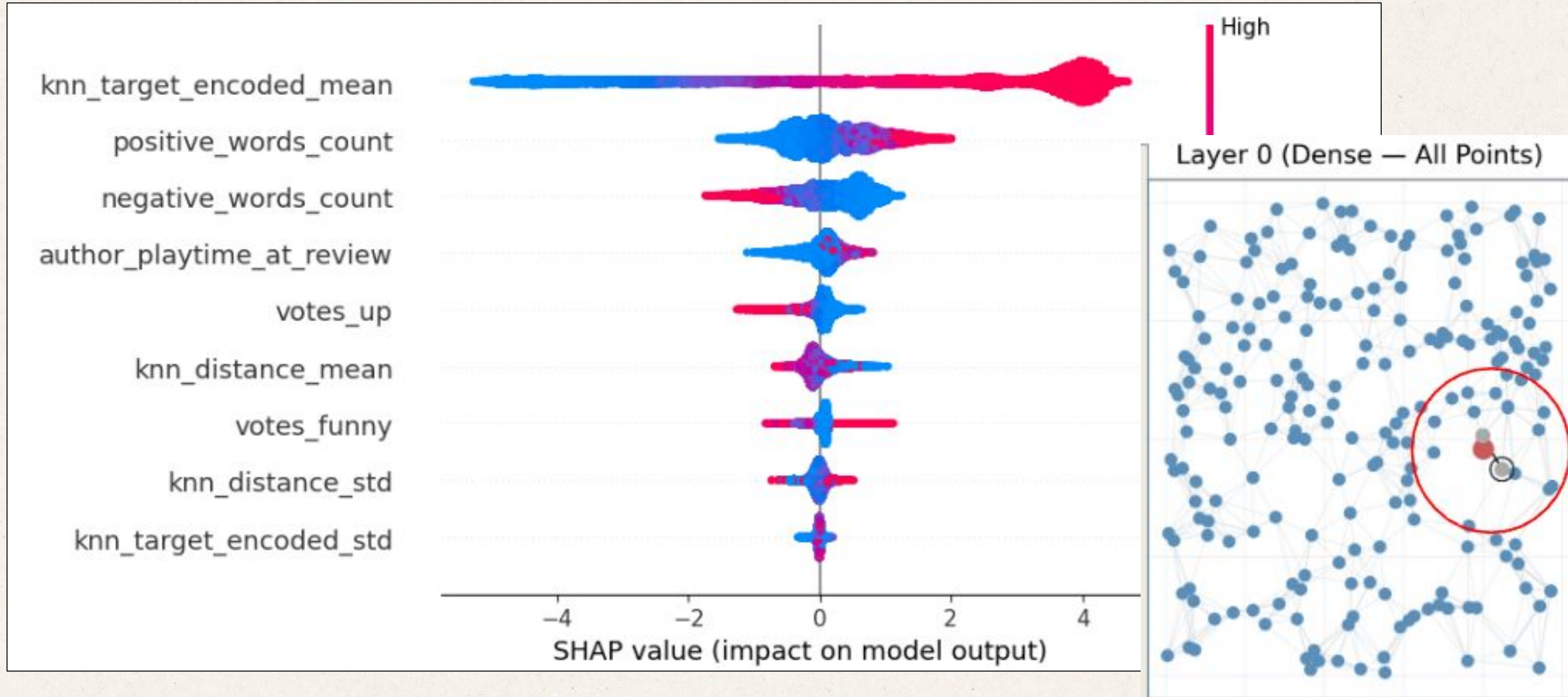
	accuracy score - test
0	0.927214
1	0.926994
2	0.928830
3	0.926498
4	0.924020
5	0.924204



Connecting BGE-M3 text embeddings with HNSW and C-TF-IDF

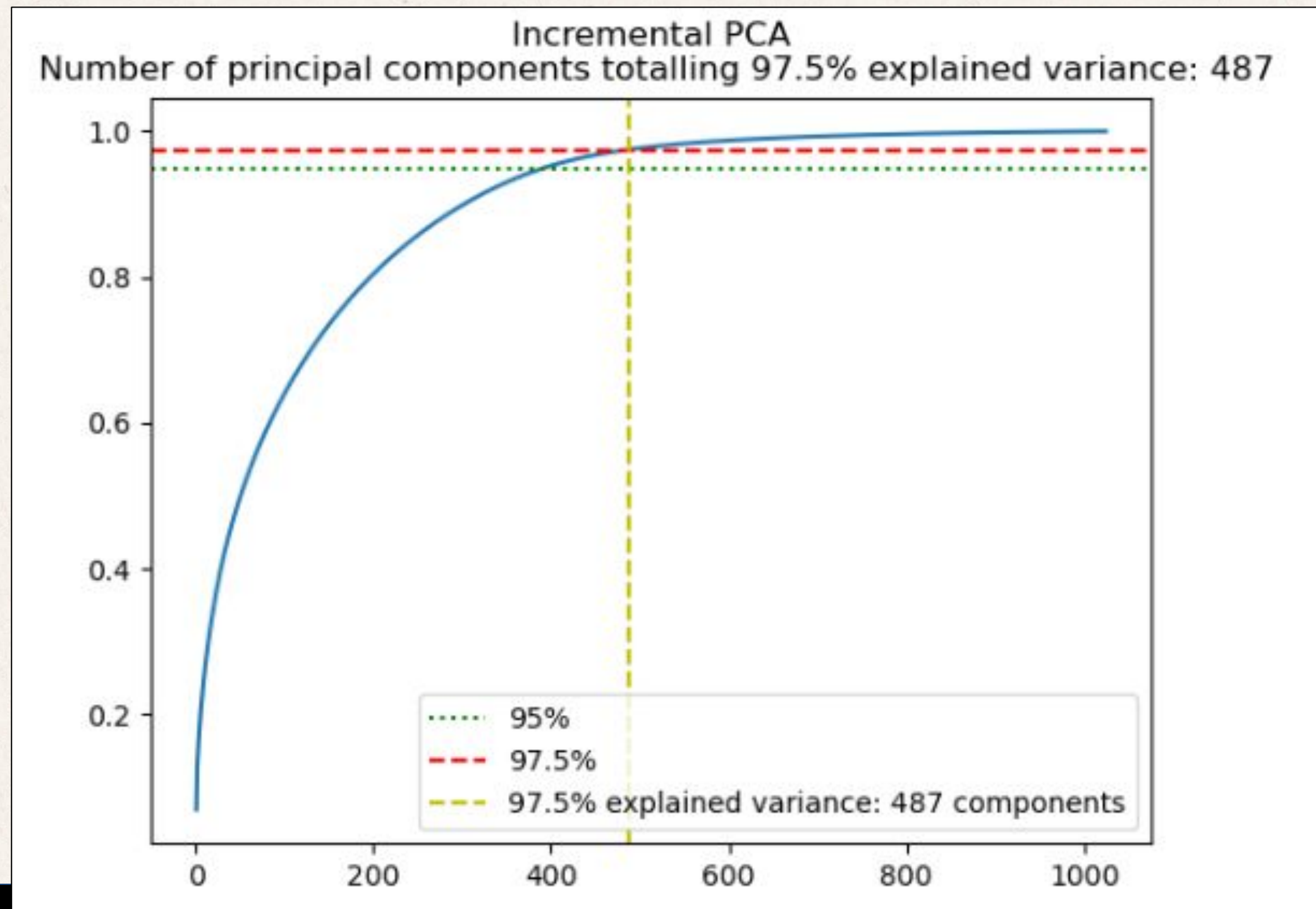
Results

XGBoost Shap Values

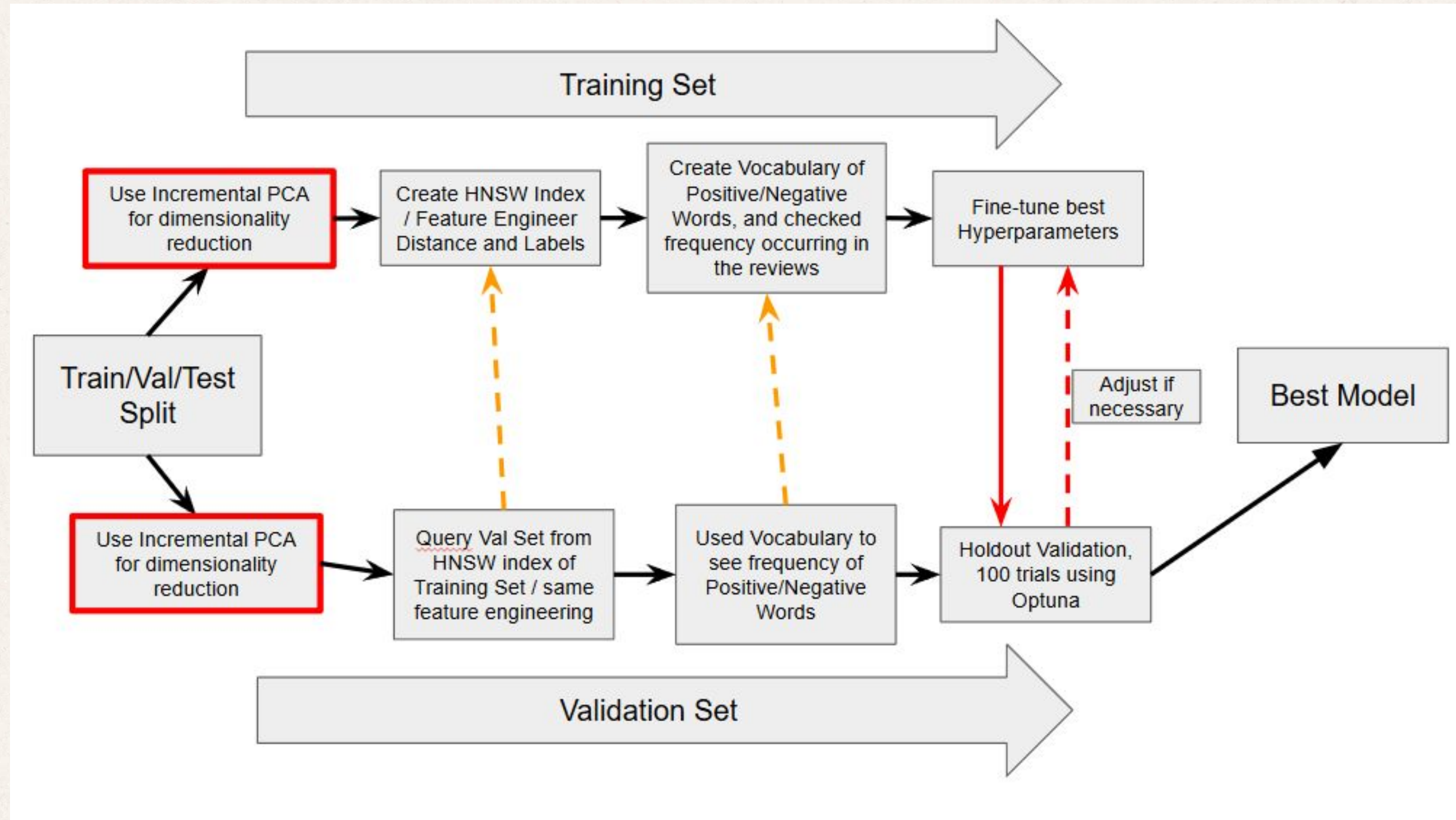


Reducing dimensions with Incremental PCA

If the problem is the number of dimensions, why not use **PCA**?



Reducing dimensions with Incremental PCA



Reducing dimensions with Incremental PCA

Results

```
score_test_df.to_parquet("score_test_df_BGE_M3_IPCA.parquet")
print(score_test_df)
```

	model	macro f1 score - test \
0	MLPClassifier	0.892907
1	HistGradientBoostingClassifier	0.892517
2	XGBClassifier	0.892402
3	LGBMClassifier	0.892172
4	LogisticRegression	0.886627
5	GaussianNB	0.885679

	accuracy score - test
0	0.920587
1	0.919908
2	0.919504
3	0.918917
4	0.912933
5	0.913355

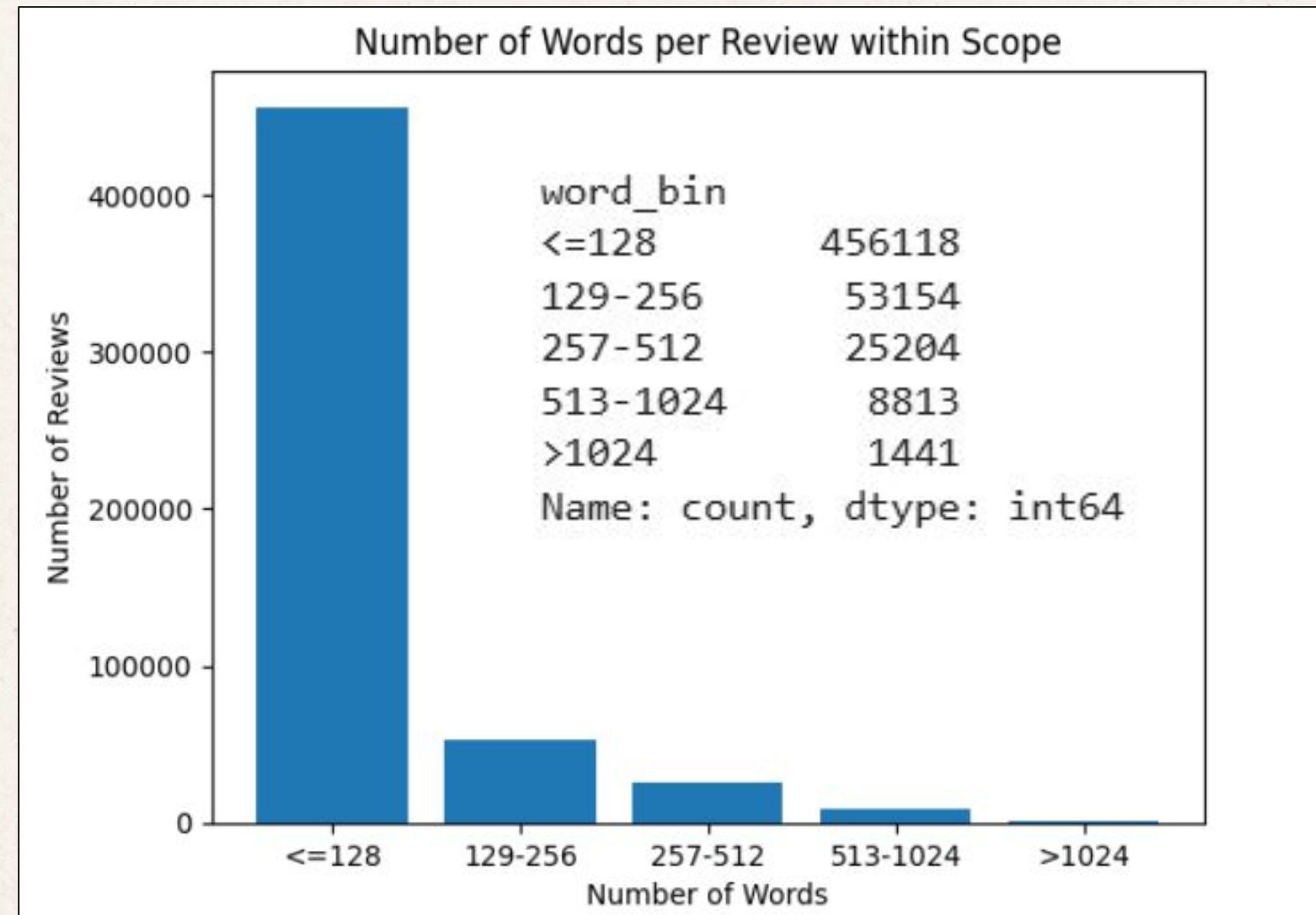
- MLPClassifier scored highest with 89.2907% macro-averaged f1-score.
- There was a 1-2% drop in performance across all models if IPCA is to be used in the pipeline.
- The time saved creating the index, however, is lost from the fit_transform method of IPCA.

Fine-tuning embedding models and creating my own text embeddings

If the problem is the number of dimensions, why not use **lighter embedding models**?

Selected Models

- **BGE-small-EN-v1.5**
 - 33.4 million parameters
 - 384 output dimensions
 - 512 max tokens
- **all-MiniLM-L6-v2**
 - 22.7 million parameters
 - 384 output dimensions
 - 256 max tokens



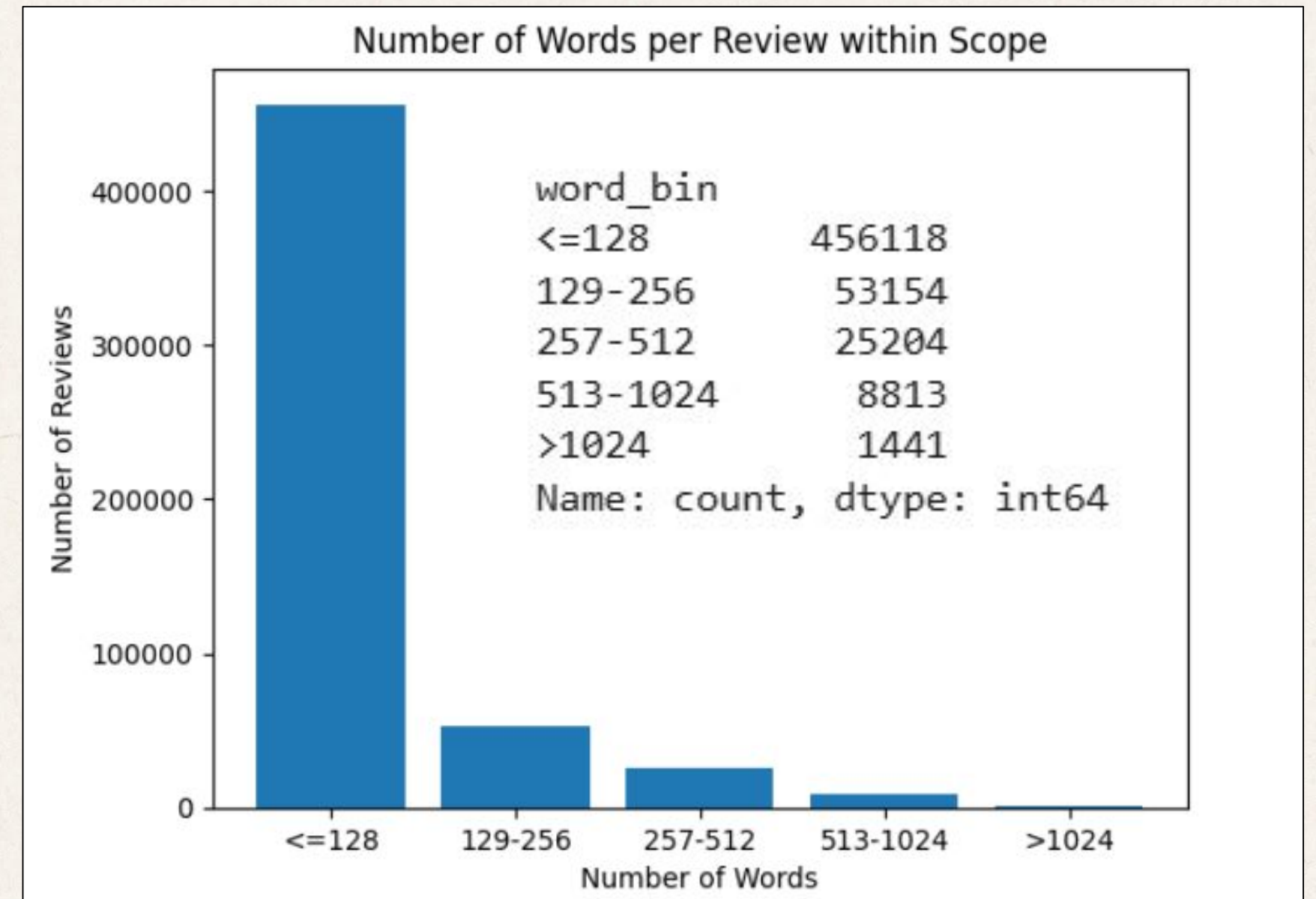
Fine-tuning embedding models and creating my own text embeddings

Strategy for limited max tokens

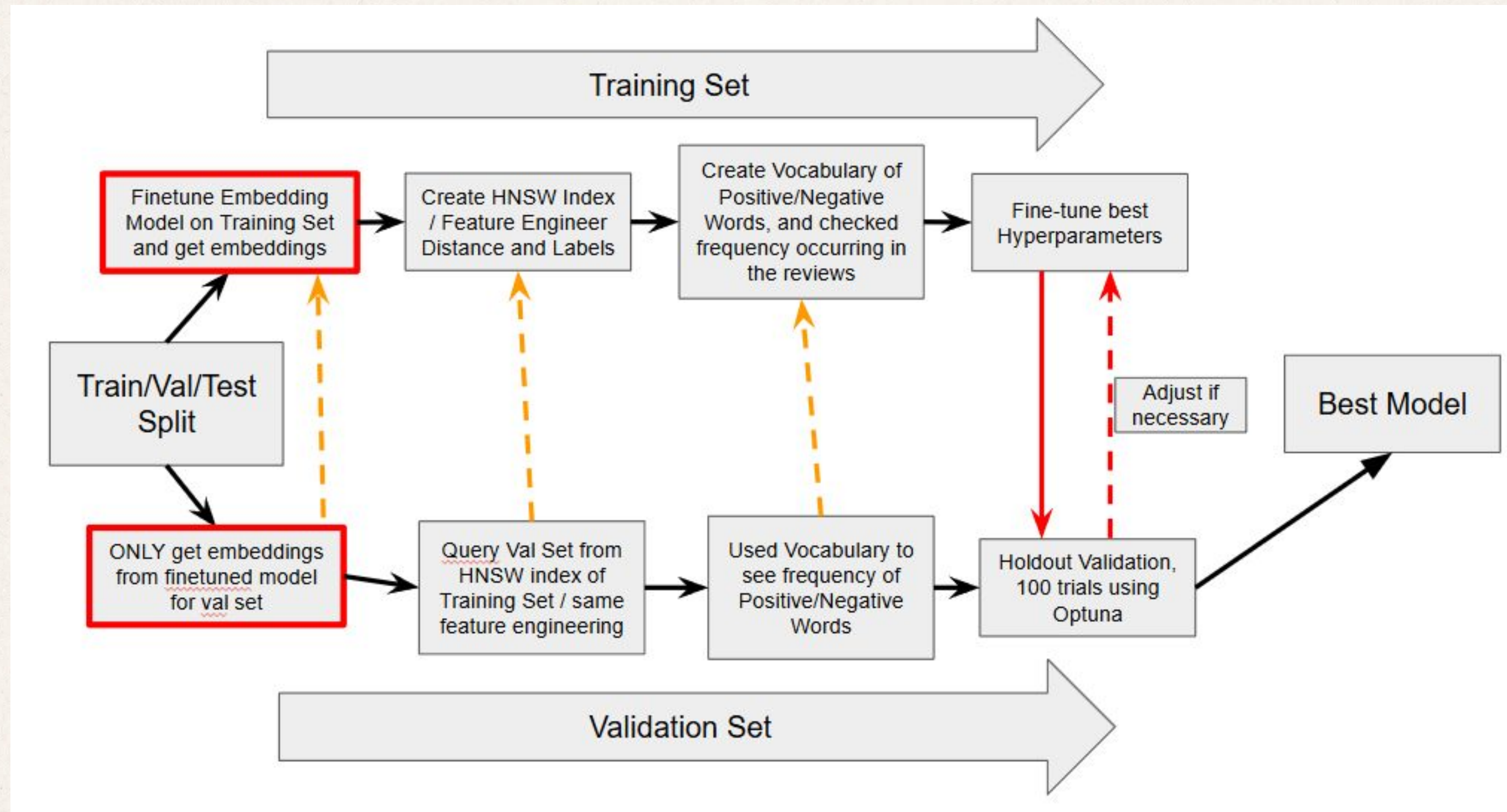
Get the tokens of the first part and last part of the review!

Ex. **256** max length

- 128 tokens first part of review
- 128 tokens last part of review



Fine-tuning embedding models and creating my own text embeddings



Fine-tuning embedding models and creating my own text embeddings

Disclosure: Finetuning lightweight embedding models is doable with 4 GB VRAM, but it takes time due to smaller batches. For the interest of time, I used Google Colab for finetuning models and encoding embeddings.

```
loss = BatchAllTripletLoss(model, distance_metric = BatchHardTripletLossDistanceFunction.cosine_distance, margin = 0.5)
#loss = BatchHardTripletLoss(model, distance_metric = BatchHardTripletLossDistanceFunction.cosine_distance, margin = 0.7)
```

```
training_args = SentenceTransformerTrainingArguments(
    output_dir = "/content/drive/MyDrive/For_Colab_GPU_use/1. a. Cleaner Containers/batchalltriple/training/continued_training",
    num_train_epochs = 3,
    #max_steps = max_steps - remaining_steps,
    per_device_train_batch_size = 64,
    gradient_accumulation_steps = 2,
    learning_rate = 2e-5 ,# default is 2e-5
    lr_scheduler_type = "cosine",
    warmup_steps = 0.1,
    weight_decay = 0.01, # adds L2 regularization, penalize large weights
    fp16 = True,
    logging_steps = 100,
    save_steps = 1000,
    save_total_limit= 1
)

trainer = SentenceTransformerTrainer(
    model = model,
    args = training_args,
    train_dataset = X_train_finetune,
    loss = loss
)
```


Fine-tuning embedding models and creating my own text embeddings

Embedding Similarity Scores

BGE-small finetuned

```
Finetuned BGE-small Score: {'steam_bge_small_embedding_finetune_eval_cosine_accuracy': 0.8813699040767387,  
                             'steam_bge_small_embedding_finetune_eval_cosine_ap': 0.9469265058802249,  
                             'steam_bge_small_embedding_finetune_eval_cosine_mcc': 0.7620041019513374}
```

***AP = Average Precision**

****MCC = Matthew's Correlation Coefficient**

Fine-tuning embedding models and creating my own text embeddings

Embedding Similarity Scores

MiniLM-L6 finetuned

```
Finetuned MiniLM-L6 Score: {'steam_minilm_L6_finetune_eval_cosine_accuracy': 0.8537170263788969,  
'steam_minilm_L6_finetune_eval_cosine_ap': 0.9254904495385775,  
'steam_minilm_L6_finetune_eval_cosine_mcc': 0.7072850269373436}
```

***AP = Average Precision**

****MCC = Matthew's Correlation Coefficient**

Fine-tuning embedding models and creating my own text embeddings

Embedding Similarity Scores

BGE-M3 OOTB

```
Baseline BGE-M3 Score: {'steam_bge_m3_baseline_finetune_eval_cosine_accuracy': 0.6330185851318945,
```

```
'steam_bge_m3_baseline_finetune_eval_cosine_ap': 0.6784812565004921,
```

```
'steam_bge_m3_baseline_finetune_eval_cosine_mcc': 0.1215218842006049}]
```

***AP = Average Precision**

****MCC = Matthew's Correlation Coefficient**

Fine-tuning embedding models and creating my own text embeddings

Baseline Classification Scores

1. Finetuning smaller models yields better results
2. Embedding similarity is a good predictor of downstream classification
3. Class-level separability does not require within-class separability.
*Note that the BGE-M3 captured entire review while the others use only first and last tokens

BGE-small finetuned

Establishing baseline - OOTB LightGBM on the finetuned BGE-small embeddings
0.9333695575619931

MiniLM-L6 finetuned

Establishing baseline - OOTB LightGBM on the finetuned MiniLM-L6 embeddings
0.9176832713450613

BGE-M3 OOTB

Establishing baseline - OOTB LightGBM on the BGE-M3 embeddings
0.8931558381487521

Fine-tuning embedding models and creating my own text embeddings

Results -
BGE-small

Highest: 93.49%
Lowest: 93.42%

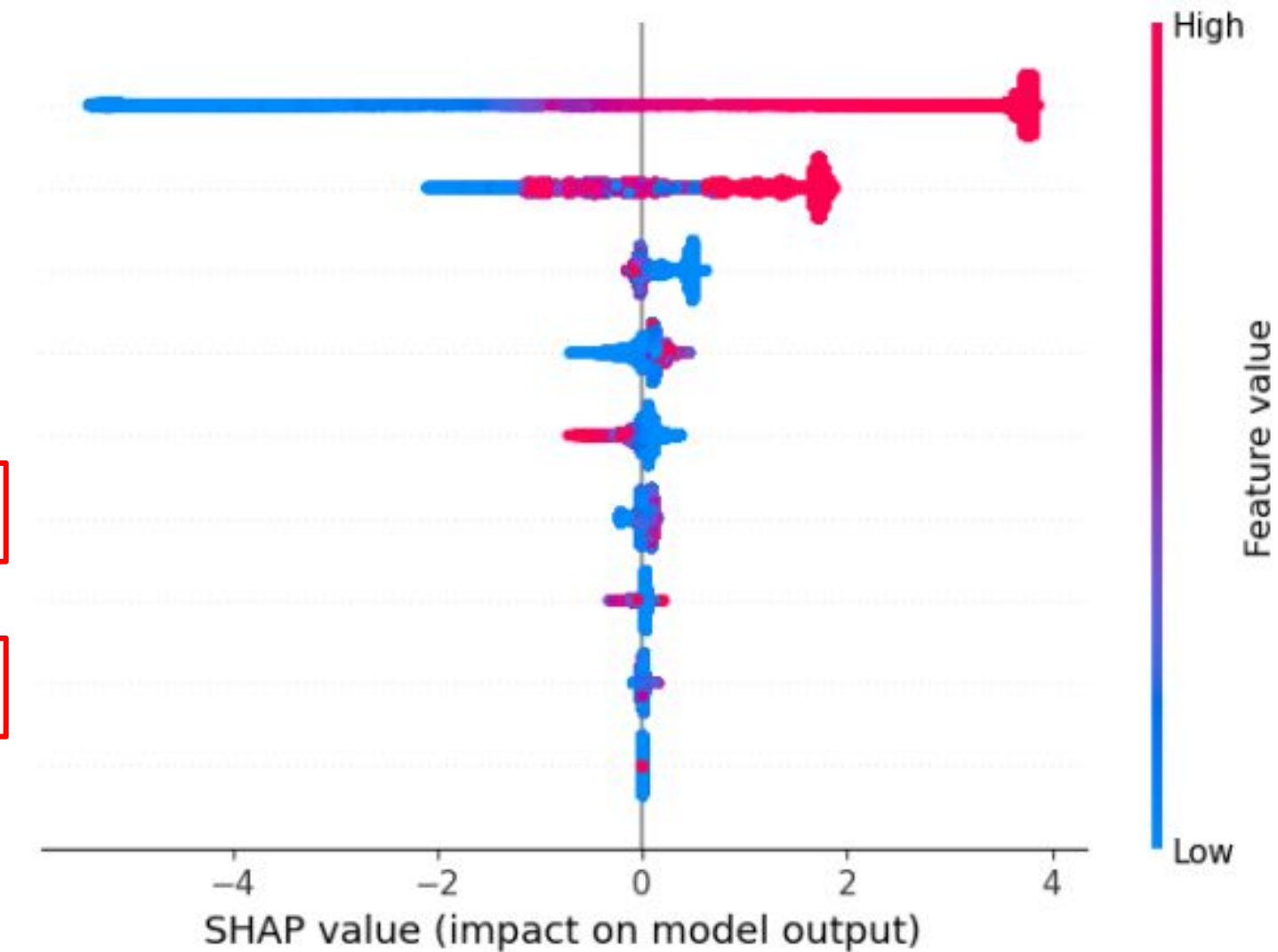
XGBoost Shap Values

```
: score_test_df.to_parquet("score_test_df_BGE_small_finetuned.parquet")  
print(score_test_df)
```

	model	macro f1 score - test \
0	LogisticRegression	0.934885
1	LGBMClassifier	0.934345
2	XGBClassifier	0.934282
3	HistGradientBoostingClassifier	0.934275
4	GaussianNB	0.934256
5	MLPClassifier	0.934173

	accuracy score - test
0	0.951592
1	0.951134
2	0.951060
3	0.951042
4	0.951262
5	0.951794

knn_target_encoded_mean
knn_distance_mean
knn_distance_std
author_playtime_at_review
votes_up
positive_words_count
votes_funny
negative_words_count
knn_target_encoded_std



Fine-tuning embedding models and creating my own text embeddings

**Results -
MiniLM-L6**

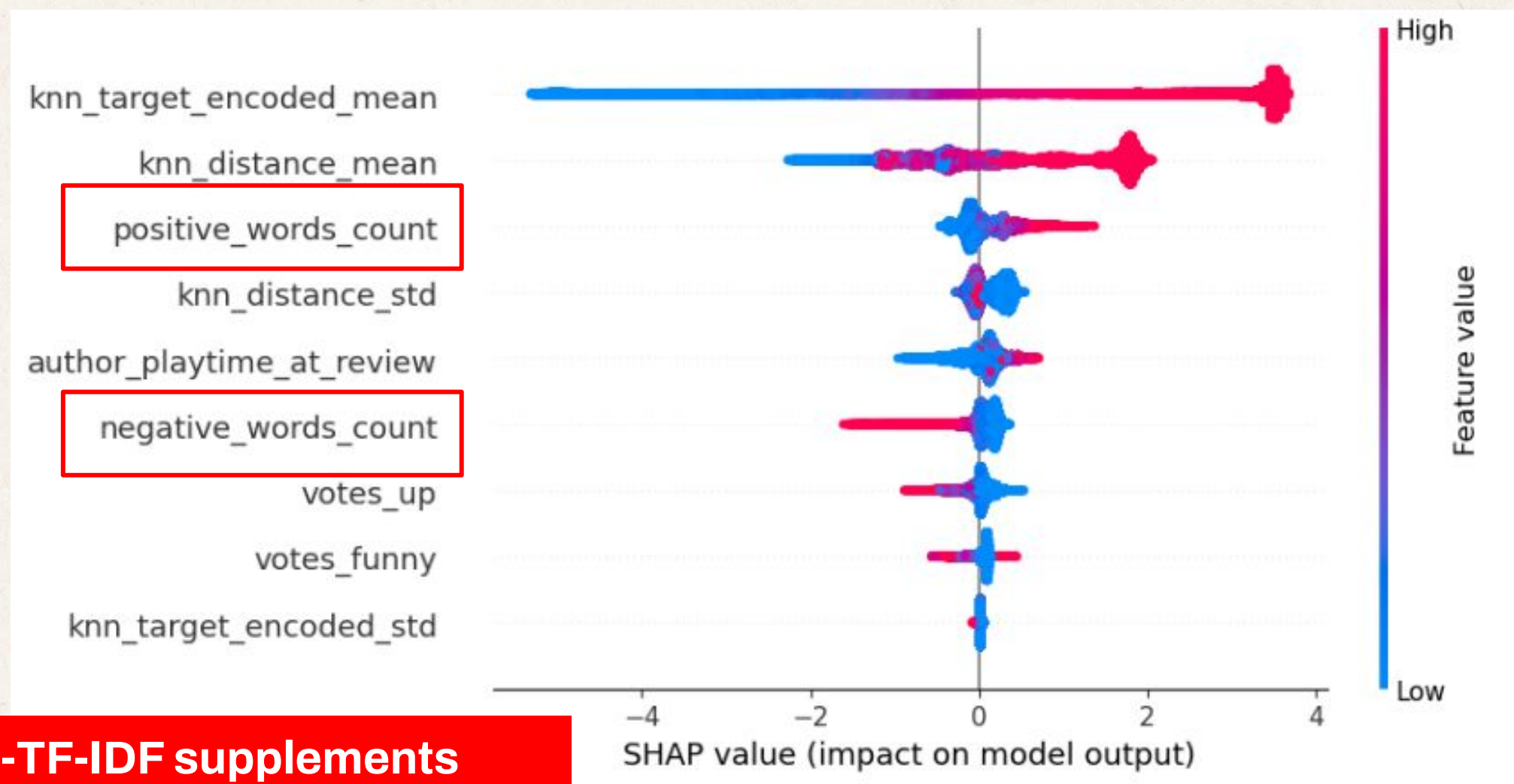
**Highest: 92.11%
Lowest: 91.74%**

XGBoost Shap Values

```
score_test_df.to_parquet("score_test_df_minilm_l6.parquet")
print(score_test_df)
```

	model	macro f1 score - test \
0	LGBMClassifier	0.921122
1	HistGradientBoostingClassifier	0.920962
2	XGBClassifier	0.920142
3	MLPClassifier	0.919328
4	LogisticRegression	0.919001
5	GaussianNB	0.917370

	accuracy score - test
0	0.941184
1	0.941166
2	0.940505
3	0.941276
4	0.939789
5	0.939679



**C-TF-IDF supplements
OOTB and domain-limited
tuned models**

Topic Modelling on Positive and Negative Reviews

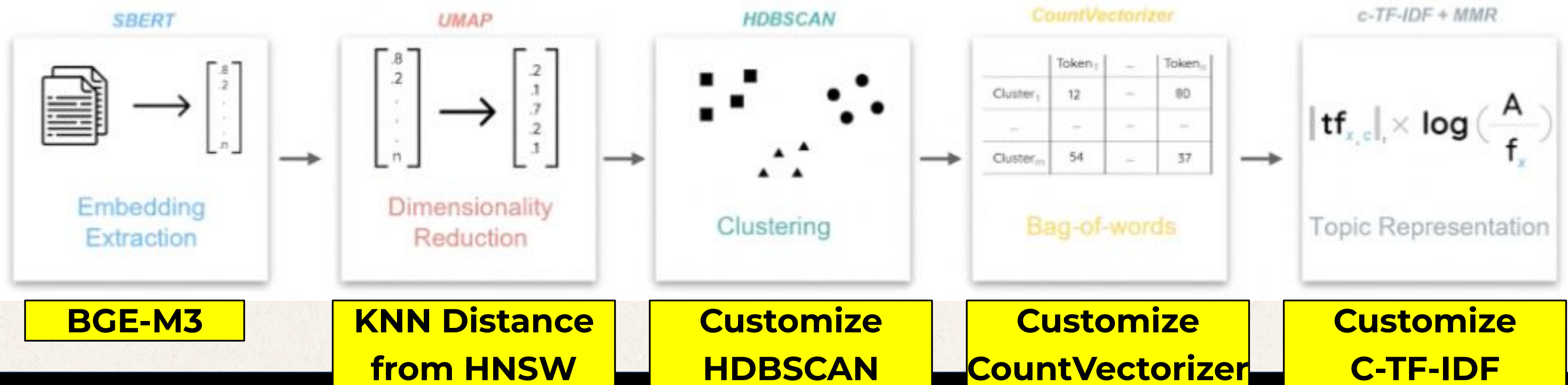


Problem # 2

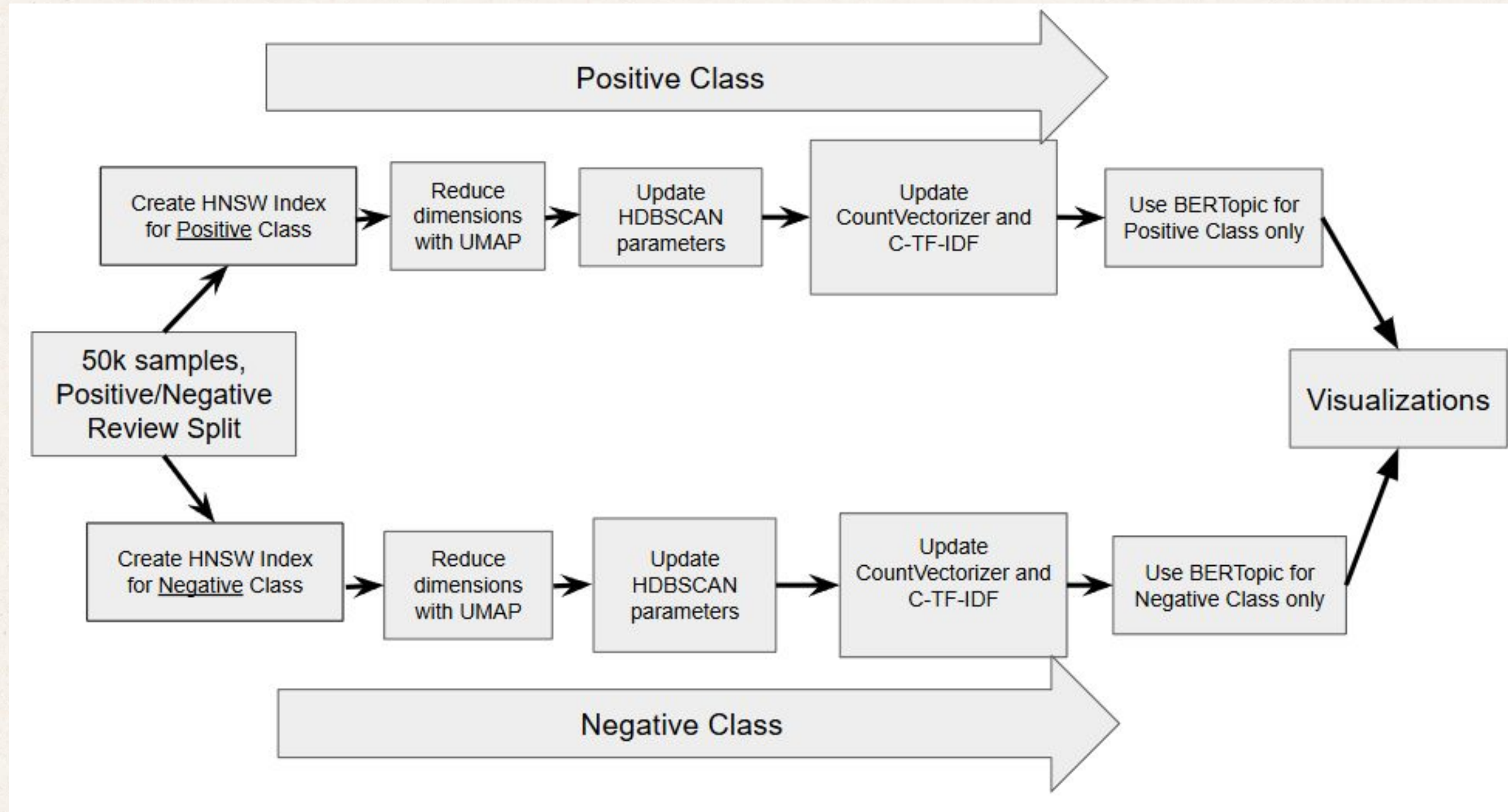
Which specific aspects of a game are driving player sentiment in these reviews?

Introducing the BERTopic Pipeline

We will sample 50,000 Positive Reviews and 50,000 Negative Reviews



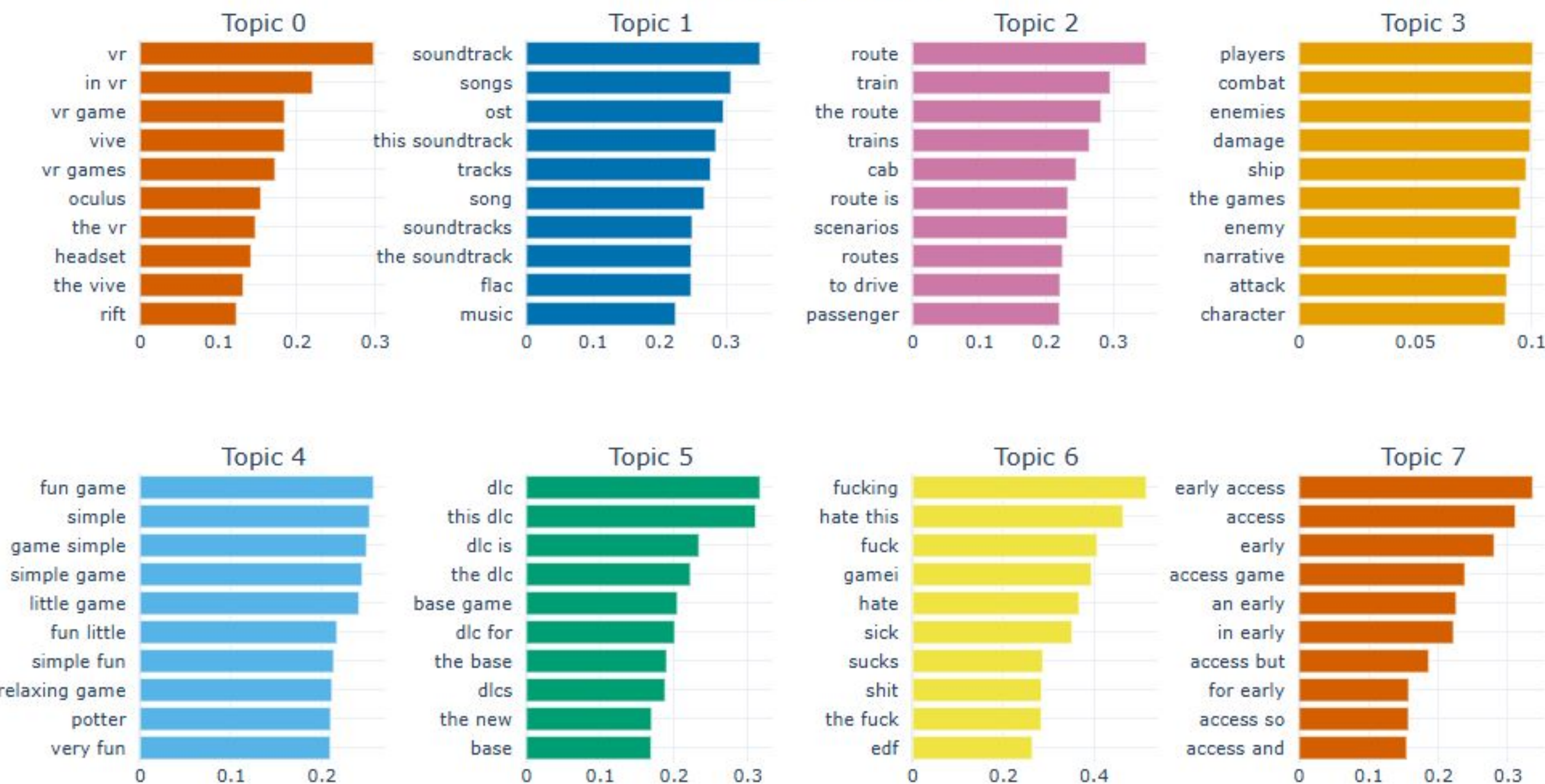
Topic Modelling on Positive and Negative Reviews



Topic Modelling on Positive and Negative Reviews

Topic Modelling - 50k Positive Reviews

Topic Word Scores



**145 total topics
excluding
outliers!**

Topic Modelling on Positive and Negative Reviews

Topic Modelling - 50k Positive Reviews



```
hate_check[hate_check].index
```

```
Index([ 51238, 346815, 397965, 403557, 454380, 188539, 446784, 25987, 22140,
        326546,
        ...,
        458202, 216143, 208502, 25092, 145001, 311584, 177177, 482428, 429125,
        6968],
      dtype='int64', length=365)
```

```
print(review_text_resampled.iloc[51238,0])
```

Haters gana hate. I'm having the best time. I would like a window option but even still no big deal.

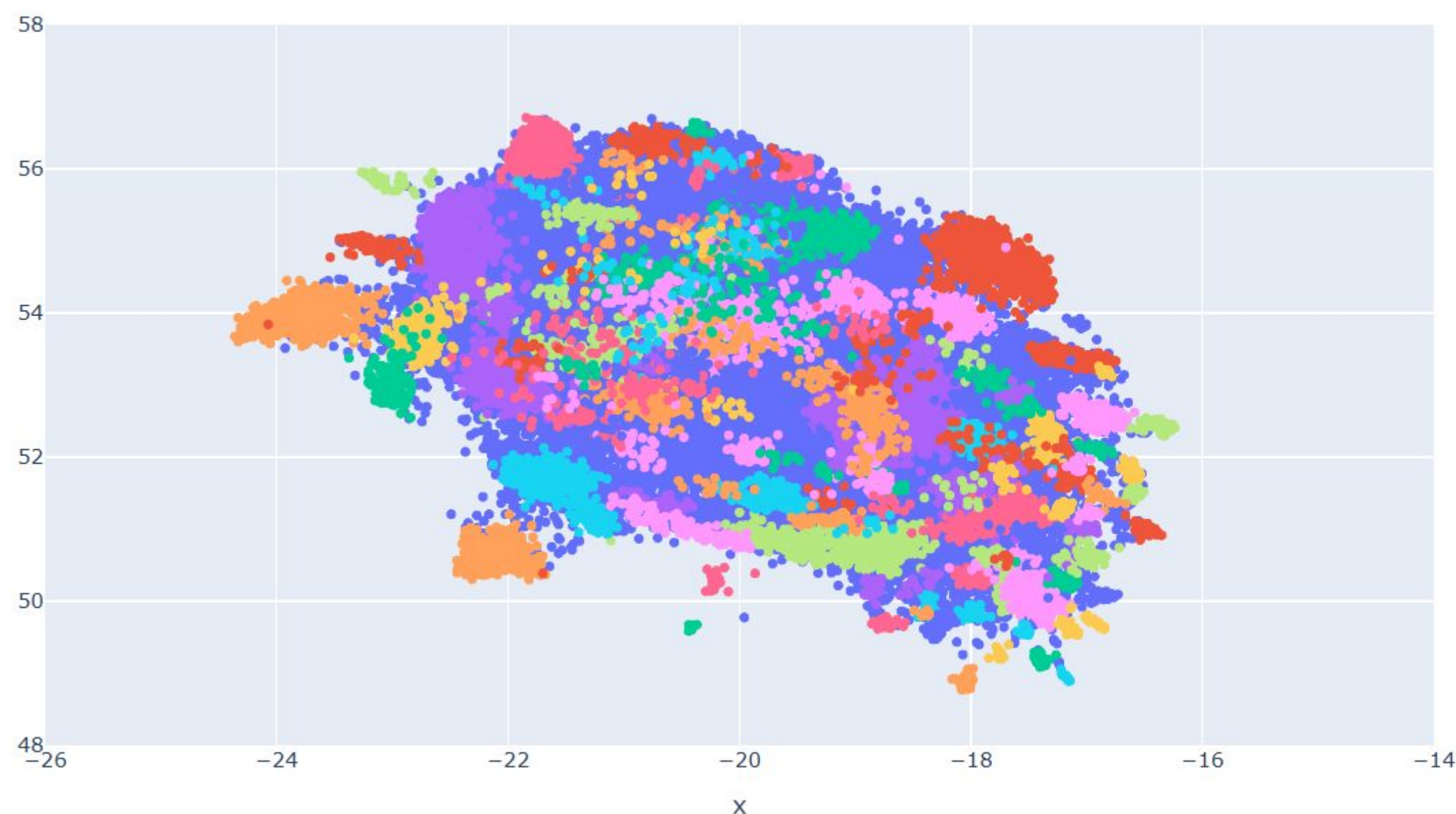
```
print(review_text_resampled.iloc[346815,0])
```

peak game
p ranks fry my dopamine receptors ten times each
i hate fork guys
pizza pasta

Topic Modelling on Positive and Negative Reviews

Topic Modelling - 50k Positive Reviews

Voted Up Review Topics



topic

- -1_puzzles_to play_levels_simple
- 18_tower_tower defense_defense_towers
- 86_tr_tr table_table tr_table
- 4_fun game_simple_game simple_simple game
- 0_vr_in vr_vr game_vive
- 66_its alright_alright its_ok its_alright
- 46_good its_good good_its good_pretty good
- 17_car_racing_cars_the car
- 16_puzzle game_puzzle_nice puzzle_great puzzle
- 39_game great_great game_game fantastic_awesome game
- 19_game love_love this_love_love it
- 35_quack_love love_overlord_captain
- 11_masterpiece_great story_all time_ever played
- 8_boring_way too_boring and_repetitive
- 2_route_train_the route_trains
- 33_hi_good game_game good_is good
- 49_weary_boobies_swing_unfair
- 6_fucking_hate this_fuck_gamei
- 75_good very_very good_good really_good so
- 48_game great_great game_great great_game awesome
- 31_roguelike_roguelite_roguelikes_rogue

Topic Modelling on Positive and Negative Reviews

Topic Modelling - 50k Positive Reviews

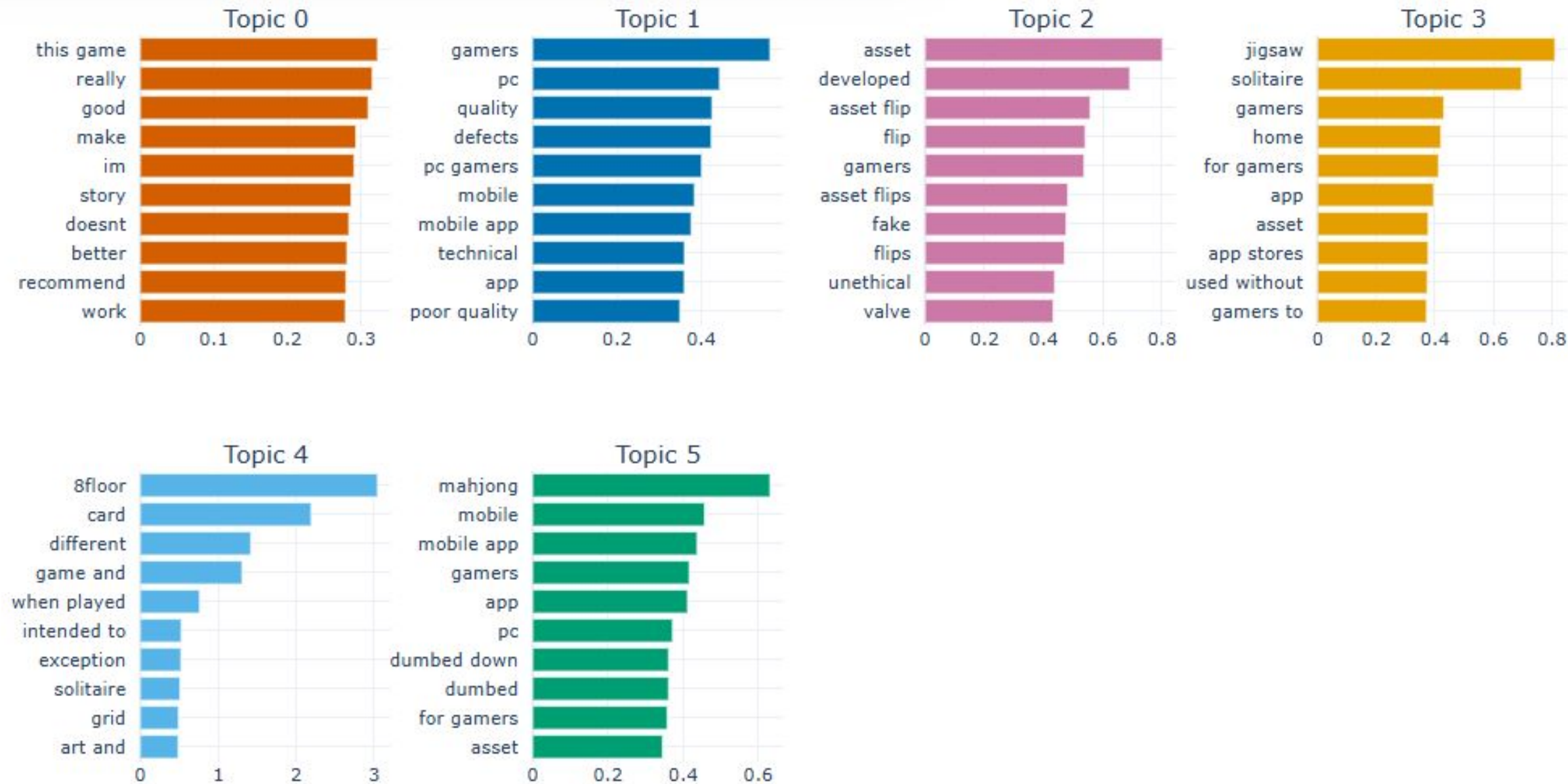
Speculations / Things to consider for Game Developers:

- Target Niches - Trains
- Common Positive Genre Combos
 - Visual Novel + Horror
 - Simple Game + Game with Friends
 - “Masterpiece” storytelling + Mafia
 - Short Game + Great Price
- Notable Complimented Game Aspects - VR, Soundtrack

Topic Modelling on Positive and Negative Reviews

Topic Modelling - 50k Negative Reviews

Topic Word Scores



6 total topics
excluding
outliers!

Topic Modelling on Positive and Negative Reviews

Topic Modelling - 50k Negative Reviews

store.steampowered.com/publisher/8floor



ARGONAUT AGENCY: THE BRONZE PLAGUE 25 Nov, 2025

“Stop the spread of bronze and save the world from plague.”

P485.00



NEW LANDS: THE SHADOW OF ALVARIA 6 Nov, 2025

“Messages in bottles become a map leading to a great mystery!”

-70% P485.00 P145.50



LOST ARTIFACTS THE GHOST OF FLORENCE 8 Oct, 2025

“Reveal the mystery of the Ghost of Florence, uncover the conspiracy, and save da Vinci's inventions!”

P485.00



GNOMES GARDEN: MASK OF THE RAT KING 23 Sep, 2025

“Help the Gnome Queen save the trolls and the Cheese Moon by fighting the Order of the Red Rose and the Rat King!”

P485.00



CHARM&CLUE 17 Sep, 2025

“Charm & Clue - an exciting detective adventure with mysteries, puzzles and secrets in the atmospheric world of the 50s-60s”

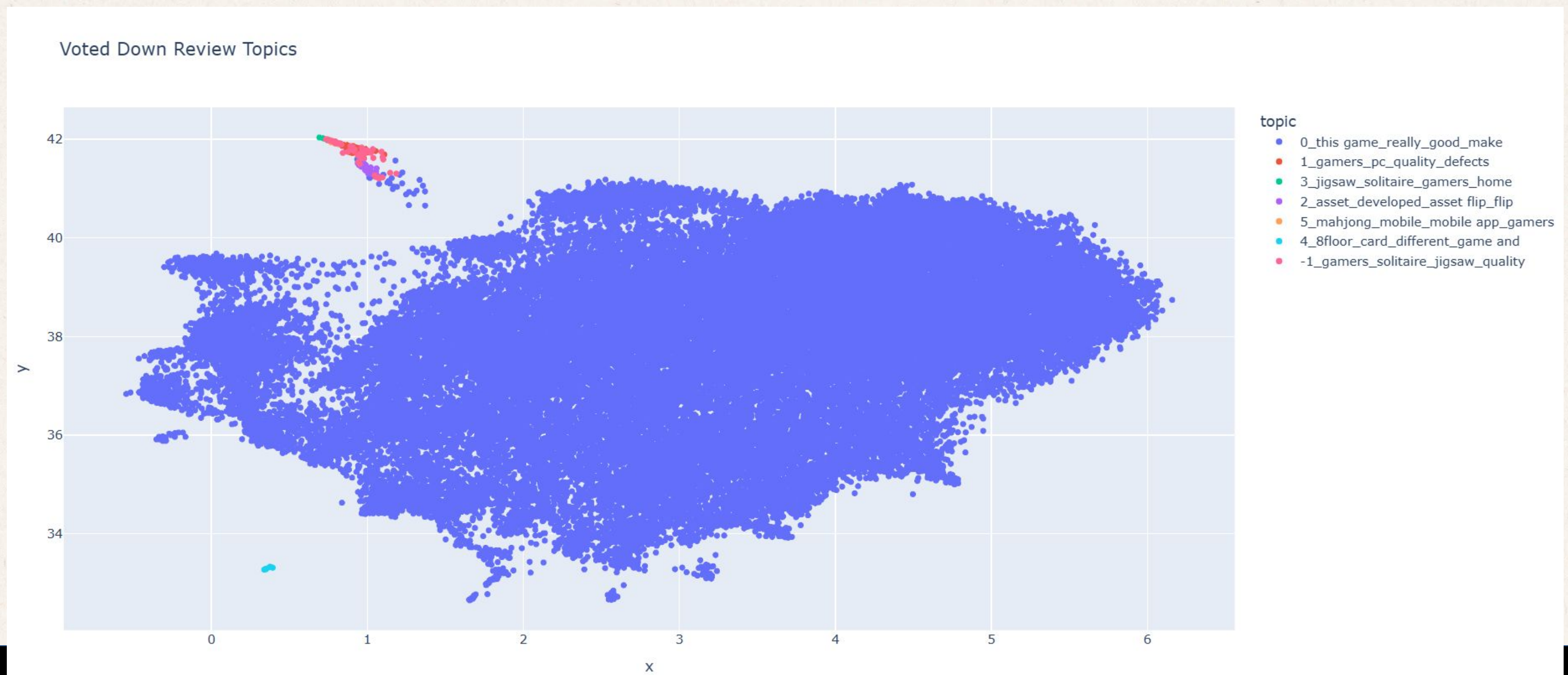
P485.00

What is 8floor?

Gaming company who uses AI generated assets in their games, recently using the same technique multiple times in a short period of time

Topic Modelling on Positive and Negative Reviews

Topic Modelling - 50k Negative Reviews



Topic Modelling on Positive and Negative Reviews

Topic Modelling - 50k Negative Reviews

Speculations / Things to consider for Game Developers:

- Avoid being egregious with AI generated assets
- Avoid low-effort and only using pre-made assets with minimal or no original work (extensive asset flipping)
- Low quality / performance will be noticed
- Negative gamer reviews are very semantically similar
 - Does not work
 - Not recommend
 - Not good
 - Bad story

Synthesis

- ✓ Addressed the embedding dimensionality problem with HNSW and lack of domain-knowledge of models with C-TF-IDF.
- ✓ Used Incremental PCA to see its impact on time and accuracy.
- ✓ Finetuned lightweight models to create my own embeddings and used it in the pipeline
- ✓ Followed BERTopic pipeline for Topic Modelling to see what drives player sentiment

Synthesis

Embedding Model	OOTB or Finetuned?	Max Tokens and Output Dimensions	Average Precision (Cosine Similarity)	Classification F1-macro with OOTB LightGBM	Best Model F1-macro
BGE-M3	OOTB	8192 tokens; 1024 dim	0.6785	89.32%	90.27% - XGBoost
all-MiniLM-L6-v2	Finetuned	256 tokens; 384 dim	0.9255	91.77%	92.11% - LightGBM
BGE-small-EN-v1.5	Finetuned	512 tokens; 384 dim	0.9469	93.37%	93.49% - Logistic Regression

Conclusion



Simulating a vector database with the training set and using HNSW index is effective in representing embeddings



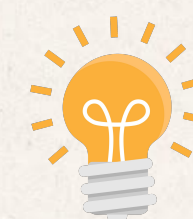
Features that can supplement domain understanding like C-TF-IDF are effective for OOTB and domain-limited models, but have diminishing returns the better the embedding similarity is.



Embedding similarity score appears to have a direct relationship with the classification score.



Class-level separability does not require within-class separability.



For processing reviews, it may be more apt to finetune lightweight embedding models than using IPCA on a larger model; Embedding model size does not guarantee better performance.

Conclusion

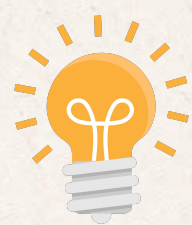


For the 2025 Steam Dataset, a finetuned BGE-small-EN-v1.5 gave the best embedding among the selected models, with a 93.49% macro-averaged F1-score using Logistic Regression, 45% threshold.



For the behavioral metrics, the implications are:

- **The lower the player's playtime is at review, more likely it will be a negative review.**
- **More players find negative reviews helpful.**
- **If a review is flagged as funny, the review can go either be positive or negative.**



For the Steam Dataset, the first and last token priority was successful not only because most of the reviews were relatively short, but because player sentiment is usually found on the first and last parts of their review.



Negative player sentiments are very semantically similar, along with low playtime at review. Positive reviews have a diverse set of topics, emphasizing where players like an aspect of a game. More information could be inferred from the scatterplot.

Thank you!